



Dissertation on

**“AI-Assisted 3D Assembly Design:
Detecting, Ranking and Generating Missing
Components in CAD Assemblies using
Graph Neural Networks”**

Submitted in partial fulfilment of the requirements for the award of degree of

Master of Technology in

Data Science and Artificial Intelligence

Project Phase - 2 (End Semester Assessment)

Submitted by:

Parthasarathy Perumal

Under the guidance of

Mr. Gaurav Siwal

Faculty, Great Learning

May 2026 – Oct 2026

DEPARTMENT OF DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

Certificate

This is to certify that the dissertation entitled “**AI-Assisted 3D Assembly Design: Detecting, Ranking and Generating Missing Components in CAD Assemblies using Graph Neural Networks**” is a bonafide work carried out by **Parthasarathy Perumal** in partial fulfilment for the completion of Project Phase 2 in the Program of Study — Master of Technology in Data Science and Artificial Intelligence, under the rules and regulations of PES University, Bengaluru, during the period May 2026 – Oct 2026.

The work reported here is original, was carried out by the candidate under our guidance, and has not been submitted elsewhere for the award of any other degree or diploma.

Mr. Gaurav Siwal
Guide,
Faculty, Great Learning

Chairperson
Dept. of DSAI, PES University

Dean of Faculty
PES University

Declaration

I hereby declare that the Project Phase 2 dissertation entitled “**AI-Assisted 3D Assembly Design: Detecting, Ranking and Generating Missing Components in CAD Assemblies using Graph Neural Networks**” has been carried out by me under the guidance of Prof. Sagarika Borah (Project Phase 1), Faculty, Great Learning, and Mr. Gaurav Siwal (Project Phase 2), Faculty, Great Learning, and submitted in partial fulfilment of the course requirements for the award of the degree of Master of Technology in Data Science and Artificial Intelligence of PES University, Bengaluru, during the academic period May 2026 – Oct 2026.

I further declare that the models, training corpus, source code and experimental results presented in this report are my own work. All external sources consulted have been cited in the bibliography. The work has not been submitted, in whole or in part, for the award of any other degree at this or any other institution.



Parthasarathy Perumal

M.Tech, Data Science and Artificial Intelligence

PES University, Bengaluru

Date: _____

Acknowledgement

I owe a considerable debt to the people who made this project possible.

My thanks go to Prof. Sagarika Borah, who guided Project Phase 1. Her insistence that a result is only as trustworthy as the data underneath it shaped how this project was run; several of the findings reported in Chapter 6 exist because that habit of auditing the corpus was established early. The octree spatial-partitioning idea that became this project’s open-surface detector was adapted from her published work on three-dimensional mesh models.

I am equally grateful to Mr. Gaurav Siwal of Great Learning, who guided Project Phase 2 through the ranking and shape-generation work. His questions about what a metric actually measures — rather than whether it had improved — led directly to the negative-sampling correction and the per-class evaluation breakdown that are reported honestly here rather than averaged away.

I owe him the scope of this project as much as its rigour. My own plan for Phase 2 ended at NodeRanker: detect that a component was missing, rank its likely type, and stop there. It was his challenge that pushed the work further — to generate an actual three-dimensional shape for the recommended component, and to treat that generation as a novel contribution and a real Phase 2 deliverable rather than an optional extra. I was not convinced it could be built in the time available. His weekly guidance is the reason it was, and the hybrid retrieval-plus-VAE generator described in Chapter 4 exists because of that extra push.

The headline detection result owes him the same debt. For most of this project the encoder sat close to chance — a mean AUC-ROC in the 0.50 range, far short of the 0.85 target — and it stayed there through run after run. His direction was consistent, and in hindsight plainly right: give the model more signal per body, and more assemblies to learn from. Widening the node feature vector and expanding the corpus to 749 curated models is what finally moved it. Runs R1 through R38 never reached the target. R39 cleared every metric at once, at a mean AUC-ROC of 0.9206. I am grateful for the patience that took, and for the weekly insistence that the ceiling was in the data rather than in the architecture.

I thank the Department of Data Science and Artificial Intelligence and the project coordinators at PES University for the structure, review milestones and feedback that kept a project of this scope moving across two semesters. The periodic reviews forced clarity at points where it would have been easier to keep accumulating experiments.

Finally, I thank my family for their patience across many long evenings of training runs, and for tolerating a machine that spent most of the last four months at full load.

Parthasarathy Perumal

Abstract

Computer-aided design packages track geometry and mating constraints with complete fidelity, but they offer no opinion about the assembly being built. They cannot tell an engineer that a bolted joint is missing its nut, suggest which component a partially-built sub-assembly most likely needs next, or propose a shape for that component. That judgement lives entirely in the designer’s experience, which makes it scarce, uneven across an organisation, and difficult to audit.

This project addresses that gap with an end-to-end graph neural network system that treats a mechanical assembly as an attributed graph: solid bodies become nodes, and genuine contact surfaces between bodies become edges. The work spans three phases sharing a single trained encoder. Phase 1 detects missing components by framing the problem as inductive link prediction over the assembly graph. Phase 2 recommends which of eight canonical component types most plausibly fills a detected gap, using a learned ranking head trained with Bayesian Personalised Ranking. Phase 3 generates an actual placeable three-dimensional shape for the recommended component through a hybrid generator that prefers retrieving a real part from a bank of training geometry and falls back to a conditional variational autoencoder only when no adequate match exists.

The system was evaluated on a corpus assembled specifically for this project: 749 STEP-format assembly models curated evenly across seven industrial mechanical-part categories, of which 648 parsed into usable graphs containing 25,155 labelled bodies and 46,767 contact edges. Under true five-fold cross-validation, Phase 1 reaches a mean AUC-ROC of 0.9206 (± 0.0121) and mean average precision of 0.9020 (± 0.0120). Phase 2 achieves Hit@1 of 0.4093 against a corpus-wide majority-class baseline of 0.3442, with Hit@5 of 0.9101 and mean reciprocal rank of 0.6102. Phase 3 reaches a test intersection-over-union of 0.6459 with a Chamfer distance of 0.0377. All three phases clear the design targets set at the start of the project.

Beyond the three-phase pipeline, this work contributes an octree-based open-surface detector that locates candidate mating joints from raw mesh geometry without requiring a proprietary CAD kernel at inference time; a component-type frequency prior that remains useful even when a single body is uploaded with no graph context; and a natural-

language explanation layer built on GNNExplainer attributions that renders predictions as engineering-readable justifications rather than raw importance weights. Two data-quality findings surfaced through exploratory analysis performed *after* training had already succeeded are reported rather than omitted, as evidence that strong aggregate metrics are not sufficient proof that every input feature is contributing.

Keywords: Graph Neural Networks, CAD Assembly, Link Prediction, Component Recommendation, Shape Generation, Variational Autoencoder, Relational Attention, Explainable AI.

Contents

Certificate	i
Declaration	ii
Acknowledgement	iii
Abstract	v
1 Introduction	1
1.1 Background	1
1.2 Motivation	1
1.3 Problem Statement	2
1.4 Objectives	3
1.5 Scope of the Project	4
1.6 Organization of the Report	4
2 Literature Survey	5
2.1 Generative CAD Models	5
2.2 Graph Neural Networks for Structural and Link-Level Reasoning	5
2.3 CAD and Mechanical Part Datasets	6
2.4 Interpretability	7
2.5 Ranking and Generative Objectives	7
2.6 Research Gap and Summary	8
3 System Requirements Specification	9
3.1 Introduction	9

3.1.1	Project Scope	9
3.2	Product Perspective	9
3.2.1	Product Features	10
3.2.2	Assumptions, Dependencies and Constraints	10
3.3	Functional Requirements	11
3.4	Non-Functional Requirements	11
3.5	System Environment	11
3.5.1	Hardware Requirements	11
3.5.2	Software Requirements	11
3.6	Use Cases	11
3.7	System Context	13
4	Proposed Methodology	15
4.1	System Architecture	15
4.2	Graph Construction	16
4.3	Feature Engineering	16
4.3.1	Node Features (34 dimensions)	16
4.3.2	Edge Features (6 dimensions)	17
4.4	Shared Encoder: AssemblyGNN	17
4.5	Phase 1: Missing-Component Detection	18
4.6	Phase 2: Next-Component Recommendation	19
4.7	Phase 3: Hybrid Shape Generation	19
4.7.1	Retrieval	19
4.7.2	Conditional VAE Fallback	20
4.8	Supporting Modules	20
4.8.1	Octree Open-Surface Detector	20
4.8.2	AssemblyTemplateDB	21
4.8.3	Explanation Layer	21
4.9	Algorithms	21
4.10	Data Flow	21
5	Implementation Details	24

5.1	Development Environment	24
5.2	Corpus Construction	24
5.3	Module Implementation	26
5.4	Training Protocol	26
5.5	Evaluation Metrics	27
5.6	Engineering Challenges	27
5.6.1	Batch-Composition Out-of-Memory	28
5.6.2	Silent Process Termination	28
5.6.3	A Promotion-Gate Comparison Bug	28
5.6.4	Component-Classification Errors	29
5.6.5	Checkpoint Write Failures and Split Reproducibility	29
6	Results and Discussion	30
6.1	Phase 1: Missing-Component Detection	30
6.2	Development History	31
6.3	Phase 2: Next-Component Recommendation	32
6.4	Phase 3: Shape Generation	33
6.5	Ablation Studies	33
6.5.1	Encoder Benchmark	33
6.5.2	Capacity Ablation	34
6.5.3	Learning Curve	34
6.5.4	Cross-Validation Protocol	35
6.5.5	Part Bank Quality Gate	35
6.6	Findings from Exploratory Data Analysis	36
6.7	Discussion	36
7	Application Interface and Demonstration	38
7.1	Application Overview	38
7.2	Sidebar Controls	39
7.2.1	Viewer Settings	39
7.2.2	Trained-Model Information	40
7.2.3	Activity Log	40

7.3	Input and Results Panels	41
7.3.1	Model Upload	41
7.3.2	Inspecting the Uploaded Assembly	42
7.4	Prediction Results	43
7.4.1	Result View	43
7.4.2	Reconstructed Components	45
7.4.3	Prediction Details Panel	46
7.5	AIDA Engineering Interpretation	47
8	Testing and Validation	49
8.1	Reproducibility Protocol	49
8.2	Validation of the Graph Construction Pipeline	50
8.3	Model Validation	50
8.4	Checkpoint Promotion Gating	50
8.5	System Testing	51
9	Conclusion and Future Work	52
9.1	Conclusion	52
9.2	Limitations	53
9.3	Future Work	53
	Bibliography	55

List of Figures

4.1	System pipeline: one shared frozen encoder feeds three task heads and a live explanation layer.	15
5.1	Corpus overview. Top: a reference image of each of the seven categories. Bottom: usable assembly graphs obtained from the 107 models in each category (648 of 749 in total, 86.5%); gate valves, highlighted, have the lowest yield at 51.4%.	25
6.1	(a) Mean and best AUC across all 39 tracked training runs, with R39 (749-model corpus) highlighted. (b) R39’s five-fold validation-AUC convergence, with final test AUC per fold in the legend.	31
7.1	Full application window in its initial state: sidebar (left), Input and Results panels (centre) and the AIDA explanation panel (bottom).	39
7.2	Viewer Settings: part colour, opacity, background and camera preset. . .	39
7.3	Trained-model information: metrics of the promoted serving checkpoint and the code version the application is running.	40
7.4	Activity Log: timestamped pipeline stages.	41
7.5	(a) Input panel with the STEP/STP upload control. (b) Results panel before a file is loaded.	42
7.6	Original view of the uploaded partial tool-post assembly, before prediction. .	43
7.7	Result view of the partial tool post: Parts (blue), Not Assembled (orange) and the Suggested Shapes group (pink).	44
7.8	The same components in the Reconstructed group, coloured by component category.	45
7.9	Prediction details panel with the reconstruction banner and the six result sections.	47

7.10 AIDA panel: Gemini-generated engineering interpretation of the GNN predictions, open-surface findings and suggested shapes.	48
---	----

List of Tables

2.1	Surveyed approaches against the capabilities required by this project. . .	8
3.1	Functional requirements.	11
3.2	Non-functional requirements.	12
3.3	Hardware environment used for development and training.	13
3.4	Software environment.	13
3.5	Primary use cases.	14
3.6	External interfaces.	14
4.1	Node feature vector composition (34 dimensions).	17
5.1	Corpus composition by category (648 usable graphs from 749 models). .	25
5.2	Principal source modules.	26
5.3	Training configuration by stage.	27
6.1	Summary of results against design targets (749-model corpus, true five-fold cross-validation).	30
6.2	Phase 1 link-prediction results, true five-fold cross-validation.	30
6.3	Three consecutive development runs producing the largest single-run gains (238-model corpus revision; superseded by the 749-model results in Table 6.2).	31
6.4	Phase 2 ranking metrics against the majority-class baseline.	32
6.5	NodeRanker per-class Hit@1 on the test split, with predicted versus true class share.	32
6.6	Phase 3 shape-generation metrics.	33
6.7	Encoder benchmark. Two-fold screening against full five-fold validation. .	34
6.8	Capacity ablation outcomes.	34

6.9	Phase 1 performance against training-corpus fraction.	35
6.10	Fixed test set versus true five-fold assignment, same corpus.	35
6.11	Part bank composition before and after the quality gate.	35
7.1	Reconstruction accuracy against the original <code>Tool_Post_10</code> (17 components; mean per family, in mm).	46
8.1	Reproducibility controls.	49
8.2	End-to-end pipeline timing, typical four-body assembly.	51

Chapter 1

Introduction

1.1 Background

Mechanical assembly design in a modern CAD package — SolidWorks, CATIA, Fusion 360 — remains a manual, expert-driven activity. The software is extremely good at the bookkeeping: it maintains exact geometry, enforces mating constraints, detects interference, and regenerates dependent features when a dimension changes. What it does not do is form any opinion about the assembly it is holding.

Consider an engineer who opens a partially-defined assembly inherited from a colleague. The CAD system will faithfully display whatever bodies are present. It will not observe that a flange has bolt holes with no bolts in them, that a shaft passing through a housing has no retaining nut, or that a valve body of this particular class is normally paired with a gland plate that is absent here. A senior designer notices these things within seconds, drawing on an accumulated memory of how such assemblies are usually built. A junior engineer, or anyone auditing an unfamiliar assembly under time pressure, may not.

This asymmetry is a genuine productivity and quality problem in industrial practice. Incomplete assemblies carry no built-in mechanism that flags what is missing, and there is no standardised way to suggest what should be added. The knowledge exists, but it is held in people rather than in the model, and it does not transfer.

1.2 Motivation

The structure of the problem suggests a specific class of solution. A rigid mechanical assembly is, in a precise sense, a graph. Solid bodies are the nodes. Physical contact between two bodies — a bolt seated in a hole, a shaft running through a bushing, two

plates clamped face to face — is an edge. The geometry of each body can be summarised as a feature vector attached to its node, and the nature of each joint can be summarised as a feature vector attached to its edge.

Graph neural networks are built to learn representations over exactly this kind of irregular relational structure, and they have driven substantial recent progress in adjacent CAD problems. Systems such as BrepGen [1] and SolidGen [2] generate valid boundary-representation solids directly, and CAD-GPT [3] synthesises construction sequences from natural-language spatial reasoning.

What these systems have in common is that they generate whole shapes or whole sequences from a specification or a prior distribution. Comparatively little work targets the narrower and more directly actionable question that a working engineer actually asks: *given this partial assembly in front of me, what is missing, what should be added next, and what should it look like?* That question is the subject of this project.

A second motivation is practical deployability. A system that requires a licensed CAD kernel API at inference time is difficult to deploy and expensive to scale. This project deliberately constrains itself to operating on STEP exports — a neutral, vendor-independent format that any designer can produce — and on mesh geometry derived from them.

1.3 Problem Statement

Given a partially-built mechanical assembly supplied as a STEP file, automatically (i) detect which components are missing, (ii) recommend the component type most likely to fill each detected gap, and (iii) generate a placeable three-dimensional shape for that component — while producing an explanation a mechanical engineer can audit, and without depending on a proprietary CAD-kernel API at inference time.

Framing the problem this way raises three practical requirements that shape every design decision in this report.

First, the system must work from geometry a designer can actually produce. Training on a synthetic or hand-labelled graph would make the learning problem easier but the system useless in practice. Accepting a real STEP export instead pushes substantial engineering weight onto the feature-extraction pipeline described in Section 4.2, rather than onto the model alone.

Second, detection, recommendation and generation are three different kinds of prediction — an edge score, a categorical ranking, and a continuous three-dimensional shape respectively. Training three unrelated models from scratch would

triple both the data requirement and the compute burden for a corpus that, by the standards of large-scale shape datasets [10, 11], is modest. Sharing a single encoder across all three tasks is a deliberate response to that constraint rather than an implementation convenience.

Third, in an engineering context an incorrect but confident suggestion is arguably worse than no suggestion at all. A designer who trusts a plausible-looking hallucination may propagate an error downstream into manufacturing. This consideration motivates both the retrieval-first bias in the shape generator (Section 4.7) and the explanation layer (Section 4.8.3): a suggestion the designer cannot audit is one they should not trust.

1.4 Objectives

The project set the following objectives at the outset, with the measurable targets shown in parentheses. Chapter 6 reports performance against each.

- O1.** Construct a curated corpus of real industrial CAD assemblies, balanced across mechanical-part categories, and an automated pipeline that converts STEP files into attributed assembly graphs.
- O2.** Design and train a shared graph encoder capable of representing both the geometry of individual bodies and the physical nature of the joints between them.
- O3. Phase 1:** Detect missing components via inductive link prediction (target: mean AUC-ROC ≥ 0.85 , mean AP ≥ 0.82 under true five-fold cross-validation).
- O4. Phase 2:** Recommend the most likely missing component type through a learned ranking head (target: Hit@5 ≥ 0.70 , MRR ≥ 0.60 , and Hit@1 genuinely above the majority-class baseline).
- O5. Phase 3:** Generate a placeable 3D shape for the recommended component (target: IoU ≥ 0.35 , Chamfer distance ≤ 0.08).
- O6.** Provide interpretability: translate model attributions into engineering-readable natural-language explanations.
- O7.** Deliver a working interactive application that runs the complete pipeline against user-uploaded STEP files.

1.5 Scope of the Project

In scope. The system operates on rigid mechanical assemblies supplied in STEP (ISO 10303) format. It covers seven industrial part categories relevant to fastening and flow-control mechanisms: bench vices, C-clamps, pipe vices, gate valves, press tools, tool posts and crane hooks. Component types are classified into eight canonical classes — long shaft, short shaft, thick plate, thin plate, bolt, washer, nut and body. The deliverable includes a trained three-phase model, a FastAPI backend, and a Streamlit front end that runs the complete detect–rank–generate–explain pipeline live.

Out of scope. The project does not address kinematic simulation, tolerance analysis, stress or thermal analysis, manufacturing process planning, or automatic assembly-sequence generation. It does not perform constraint solving or write mating constraints back into a native CAD file. The conditional-VAE generative fallback is retained as a safety net rather than a validated component: the offline evaluation never invoked it (Section 9.2). Non-rigid components such as gaskets, springs under load, cables and hoses are not modelled. The corpus is domain-specific to fastening and flow-control mechanisms and no claim is made about generalisation beyond that domain.

1.6 Organization of the Report

Chapter 2 surveys related work across generative CAD modelling, graph representation learning, mechanical part datasets and model interpretability, and identifies the specific gap this project addresses. **Chapter 3** states the system requirements — functional, non-functional, hardware and software. **Chapter 4** presents the proposed methodology: the graph construction pipeline, the feature engineering, the shared encoder, and the three task heads. **Chapter 5** covers implementation, including the corpus construction, the training protocol, the evaluation metrics, and the engineering problems encountered and solved during development. **Chapter 6** reports and discusses results for all three phases, together with the development history and two data-quality findings from exploratory analysis. **Chapter 7** walks through the deployed application using screenshots of the running system, including the reconstruction of missing components. **Chapter 8** describes testing and validation, and **Chapter 9** concludes and sets out directions for future work.

Chapter 2

Literature Survey

This chapter reviews the research this project builds on, grouped into five themes, and closes by stating the gap the project addresses.

2.1 Generative CAD Models

Recent research has concentrated on generating CAD geometry directly. BrepGen [1] applies a structured latent diffusion model to synthesise valid boundary-representation (B-Rep) solids, addressing the difficult constraint that a generated B-Rep must be topologically well-formed, not merely visually plausible. SolidGen [2] takes an autoregressive approach over B-Rep topology and geometry, generating faces, edges and vertices in sequence. CAD-GPT [3] works at a different level of abstraction altogether, synthesising CAD *construction sequences* — the ordered list of modelling operations — from natural-language spatial reasoning.

These systems are impressive and technically distinct from one another, but they share an orientation that matters here: each generates an individual part or a complete design from a specification or a prior distribution. None of them consumes a partially-built *assembly* and reasons about which specific component is absent from it. The problem addressed in this report is therefore complementary to this line of work rather than competing with it.

2.2 Graph Neural Networks for Structural and Link-Level Reasoning

Graph convolutional and attention architectures [6, 7, 8] established the modern toolkit for learning over irregular relational data. Kipf and Welling’s graph convolutional net-

work [6] introduced the spectral-approximation formulation that made deep learning on graphs practical at scale. Graph Attention Networks [7] replaced fixed neighbourhood weighting with learned attention, allowing a node to weight its neighbours by relevance. GraphSAGE [8] contributed the inductive sampling-and-aggregation framework that permits inference on graphs never seen during training — a property this project depends on directly, since the system must reason about assemblies uploaded after training has finished.

Relational variants extend this further. RGATConv [9] adds per-edge-type parameters to the attention mechanism. This is the mechanism the present work builds on most directly, for a physically motivated reason: the type of a mechanical joint — rigid, revolute, slider or cylindrical — materially changes how two connected bodies ought to influence each other’s representation. A bolt rigidly fixed into a plate and a shaft free to rotate within a bushing are topologically identical neighbour relationships but mechanically quite different, and a model that cannot distinguish them is discarding information a mechanical engineer would consider fundamental.

A recent theoretical analysis [4] raises a caution worth recording. It questions how much of the reported performance of standard link-prediction models is genuinely attributable to learned features, as opposed to simple structural heuristics such as common-neighbour counts that the model can recover trivially. The node features used in this project are dominated by affine-invariant shape descriptors rather than pure topology, which should make the encoder less susceptible to that particular failure mode — though no claim is made here to have isolated the effect experimentally.

Heterogeneous graph contrastive learning [5] offers a self-supervised alternative to the supervised link-prediction and ranking objectives adopted in this work. It was not pursued here, primarily because the corpus provides reliable supervised labels — real contact edges are directly observable from geometry — but it remains a reasonable direction for a larger, more weakly-labelled corpus.

2.3 CAD and Mechanical Part Datasets

Large open datasets have been central to progress in learned shape understanding. ABC [10] provides a very large collection of CAD models with parametric boundary representations, and PartNet [11] contributes fine-grained, hierarchical part-level segmentation.

Both, however, are dominated by consumer objects and generic geometry — chairs, lamps, household items — rather than the industrial, fastener- and joint-heavy assemblies that this system targets. A model trained predominantly on furniture has little opportunity to learn that a bolt through a flange usually implies a nut on the far side.

This mismatch is the reason the present project builds and curates its own corpus, described in Section 5.2, rather than adopting an existing benchmark. That decision cost considerable effort but was, in retrospect, unavoidable.

2.4 Interpretability

GNNExplainer [12] produces post-hoc subgraph and feature attributions for a trained graph neural network’s prediction, identifying which edges and which input features most influenced a particular output. It is used in this project as the numerical backbone of the explanation layer.

The design choice made here is what happens to those attributions afterwards. Presenting a mechanical engineer with a ranked list of feature-importance weights is not useful; the quantity being explained is not one they reason about. Section 4.8.3 describes how those scores are instead translated into engineering language — identifying a detected pattern as an incomplete fastening operation, for instance, and naming which specific joints require completion.

2.5 Ranking and Generative Objectives

The Phase 2 ranking head is trained using Bayesian Personalised Ranking (BPR) [14], an objective originally proposed for implicit-feedback recommender systems. Its adoption here rests on a structural correspondence: next-component recommendation requires ranking a small candidate set given an incomplete context, with only relative rather than absolute preference labels available per assembly. That is precisely the setting BPR was formulated for.

The Phase 3 generative fallback is a conditional variational autoencoder [16] operating on a voxel occupancy grid. The voxel representation follows that used by 3D-R2N2 [17] for single- and multi-view 3D reconstruction, though conditioned here on the frozen encoder’s assembly-context vector rather than on an image.

For completeness, shallow structure-only embedding methods such as node2vec [18] were considered and rejected. They learn node representations from random-walk co-occurrence alone, with no mechanism to consume continuous node or edge attributes. For an assembly graph whose nodes carry rich geometric descriptors and whose edges carry a physically meaningful joint-type label, discarding that information would forfeit most of the available signal.

2.6 Research Gap and Summary

Table 2.1 summarises the surveyed work against the capabilities this project requires.

Table 2.1: Surveyed approaches against the capabilities required by this project.

Work	Contribution	Consumes partial assembly?	Suggests missing part?
BrepGen [1]	Latent diffusion over B-Rep solids	No	No
SolidGen [2]	Autoregressive B-Rep synthesis	No	No
CAD-GPT [3]	Construction sequences from language	No	No
GCN / GAT / GraphSAGE [6, 7, 8]	Core graph-learning architectures	N/A	N/A
RGATConv [9]	Per-edge-type relational attention	N/A	N/A
ABC [10], PartNet [11]	Large CAD/part datasets	N/A	No
GNNExplainer [12]	Post-hoc GNN attribution	N/A	N/A
This work	Detect, rank and generate missing components	Yes	Yes

The gap is clear enough to state in one sentence. Generative CAD research produces new geometry but does not reason about incompleteness; graph learning research provides the architectural tools but has not been applied to mechanical assembly completion on real STEP geometry; and the available part datasets do not cover the industrial assembly domain where the problem actually arises. This project sits at the intersection of the three: it applies relational graph learning to a purpose-built industrial corpus in order to detect, rank and generate missing assembly components, and it explains its own predictions.

Chapter 3

System Requirements Specification

3.1 Introduction

This chapter specifies what the system must do, the constraints under which it must operate, and the environment required to build and run it. The requirements were fixed early in Project Phase 1 and revised once, at the Phase 2 boundary, when shape generation was added as a third phase.

3.1.1 Project Scope

The software accepts a STEP-format assembly file, converts it into an attributed graph, and runs three inference stages over a shared encoder to detect missing components, recommend a replacement component type, and generate a placeable 3D shape for it. Results are presented through a browser interface together with a natural-language explanation. The system is designed for single-user interactive operation on a workstation rather than for multi-tenant deployment.

3.2 Product Perspective

The system is self-contained and does not integrate into any CAD package’s plugin architecture. It consumes the neutral STEP interchange format, which every major CAD system can export. This decision keeps the system vendor-independent and avoids any licensed-API dependency at inference time, at the cost of losing access to the originating system’s parametric feature tree.

3.2.1 Product Features

- STEP file upload and automated parsing into an assembly graph.
- Missing-component detection with per-candidate confidence scores.
- Next-component type recommendation with ranked alternatives.
- 3D shape generation for the recommended component, exportable as a mesh.
- Octree-based open-surface detection indicating *where* on the model a component is likely missing.
- Natural-language explanation of each prediction.
- Interactive 3D visualisation of the uploaded assembly and the generated component.

3.2.2 Assumptions, Dependencies and Constraints

- **Assumption:** Uploaded assemblies are rigid-body mechanical assemblies within, or close to, the seven trained categories.
- **Assumption:** Bodies in the STEP file are geometrically distinct solids that can be fragmented and separated by the parsing kernel.
- **Dependency:** The `gmsh`/OpenCASCADE geometry kernel must be able to parse the supplied file within the configured timeout.
- **Dependency:** Natural-language explanation requires a valid `GEMINI_API_KEY`; the system degrades gracefully to numeric attributions without it.
- **Constraint:** Inference runs on a single Apple-Silicon GPU via the Metal Performance Shaders backend; no CUDA dependency exists.
- **Constraint:** The conditional-VAE fallback operates on a 32^3 voxel grid, which bounds achievable geometric detail.

Table 3.1: Functional requirements.

ID	Requirement
FR-1	The system shall accept a STEP (.step/.stp) file uploaded through the web interface.
FR-2	The system shall fragment the file into individual solid bodies and construct an attributed graph with one node per body.
FR-3	The system shall create an edge between two bodies only where their boundary face sets physically intersect after fragmentation.
FR-4	The system shall extract a 34-dimensional geometric feature vector for every node and a 6-dimensional feature vector for every edge.
FR-5	The system shall classify each body into one of eight canonical component types.
FR-6	The system shall score candidate body pairs and flag probable missing components (Phase 1).
FR-7	The system shall rank the eight candidate component types for a detected gap and return them in order of likelihood (Phase 2).
FR-8	The system shall generate a 3D mesh for the top-ranked component, preferring retrieval from the part bank over generation (Phase 3).
FR-9	The system shall identify candidate open mating surfaces using octree spatial partitioning.
FR-10	The system shall produce a natural-language explanation of each prediction.
FR-11	The system shall render the uploaded assembly and the generated component in an interactive 3D viewer.
FR-12	The system shall allow the generated mesh to be exported for downstream use.

3.3 Functional Requirements

3.4 Non-Functional Requirements

3.5 System Environment

3.5.1 Hardware Requirements

3.5.2 Software Requirements

3.6 Use Cases

Four primary use cases drive the interface design. Table 3.5 summarises them; the two most important are described below in full.

UC-1: Audit an inherited assembly. The designer uploads a STEP file they did not author. The system parses it, builds the assembly graph, scores every non-adjacent body pair, and returns those whose best candidate connection scores anomalously low relative to structurally similar peers. In parallel, the octree detector reports clusters of

Table 3.2: Non-functional requirements.

ID	Attribute	Requirement
NFR-1	Performance	The complete detect $\rightarrow$ rank $\rightarrow$ generate $\rightarrow$ explain pipeline shall finish within approximately 30 seconds for a typical four-body assembly.
NFR-2	Reliability	Parsing failures shall be caught and reported without terminating the session; a bounded-timeout retry policy shall apply.
NFR-3	Reproducibility	Cross-validation splits shall be seeded per fold and per graph so that reported metrics are exactly reproducible from the same corpus snapshot.
NFR-4	Portability	The system shall run on macOS with Apple Silicon without requiring CUDA or a licensed CAD kernel.
NFR-5	Usability	Predictions shall be accompanied by an explanation in mechanical-engineering vocabulary rather than raw attribution weights.
NFR-6	Maintainability	All hyperparameters and data paths shall reside in a single configuration file rather than being embedded in source code.
NFR-7	Resource bounds	Training shall remain within the 20.13 GB Metal Performance Shaders allocation ceiling of the development machine.
NFR-8	Graceful degradation	Absence of the language-model API key shall disable explanations only, leaving all numeric predictions intact.

unmated free surface. The designer sees both signals, and the interface distinguishes between locations where the two agree — a strong claim — and where only one fires.

The precondition is a parseable STEP file containing at least two distinct solid bodies. The main alternative flow is a parse failure: the bounded-timeout policy aborts, and the system reports which stage failed rather than silently returning an empty graph.

UC-2: Complete a detected gap. Having accepted a detection from UC-1, the designer requests a completion. The ranking head returns the eight component types in likelihood order. For the top-ranked type the hybrid generator queries the part bank; if a match clears the type-dependent threshold, that real mesh is returned, and if not the conditional VAE produces a novel shape. The result is displayed in the 3D viewer and can be exported.

The postcondition worth noting is that the system always reports *which* strategy produced the geometry. A retrieved part and a generated one carry different confidence, and collapsing that distinction would mislead the user in exactly the situation where the stakes are highest.

Table 3.3: Hardware environment used for development and training.

Component	Specification
Machine	Apple MacBook Pro, Apple Silicon (M-series)
Processor	Apple Silicon SoC with unified memory architecture
System memory	16 GB unified
GPU acceleration	Metal Performance Shaders (MPS) backend, 20.13 GB allocation ceiling
Storage	Approximately 40 GB for corpus, processed graphs, part bank and checkpoints
Minimum to run inference	8 GB RAM; GPU optional (CPU inference supported, slower)

Table 3.4: Software environment.

Component	Specification
Operating system	macOS (Darwin 25.x)
Language	Python 3.12.5
Deep learning	PyTorch 2.12.0 with MPS backend
Graph learning	PyTorch Geometric 2.7.0 [13]
Geometry kernel	gmsk with OpenCASCADE
Mesh processing	trimesh
Backend service	FastAPI
Front end	Streamlit
Explanation agent	Google Gemini API
Environment management	uv, Python virtual environment

3.7 System Context

The system occupies a deliberately narrow position in a designer’s workflow. It does not replace the CAD package, integrate into its plugin architecture, or write back into a native file format. It consumes a neutral STEP export and returns advice plus optional geometry.

This narrowness is a design position rather than a limitation of effort. Writing mating constraints back into a native CAD file would require a licensed kernel API at inference time, which the project explicitly set out to avoid.

Table 3.5: Primary use cases.

ID	Use case	Primary actor’s goal	Phases used
UC-1	Audit an inherited assembly	Determine whether anything is missing from a partially-defined assembly	1, supporting
UC-2	Complete a detected gap	Obtain a component-type recommendation and a placeable shape for it	1, 2, 3
UC-3	Reason about a single body	Get a plausible next-component suggestion with no surrounding assembly context	Template prior
UC-4	Justify a suggestion	Understand <i>why</i> the system proposed a particular component	Explanation layer

Table 3.6: External interfaces.

Interface	Direction	Content
STEP upload	Inbound	ISO 10303 assembly file from any CAD package
Mesh export	Outbound	Generated or retrieved component geometry
Language model API	Outbound	Structured attribution bundle for explanation
Part bank	Internal	Indexed geometry from the training corpus
Model checkpoints	Internal	Encoder, ranking head, shape VAE

Chapter 4

Proposed Methodology

4.1 System Architecture

Figure 4.1 summarises the complete pipeline. A STEP file is converted into an attributed graph, encoded once by a shared relational-attention network, and processed by three task heads. Two supporting modules — the AssemblyTemplateDB prior and the octree open-surface detector — operate independently of the encoder and contribute their evidence directly at the explanation layer alongside the task heads’ outputs.

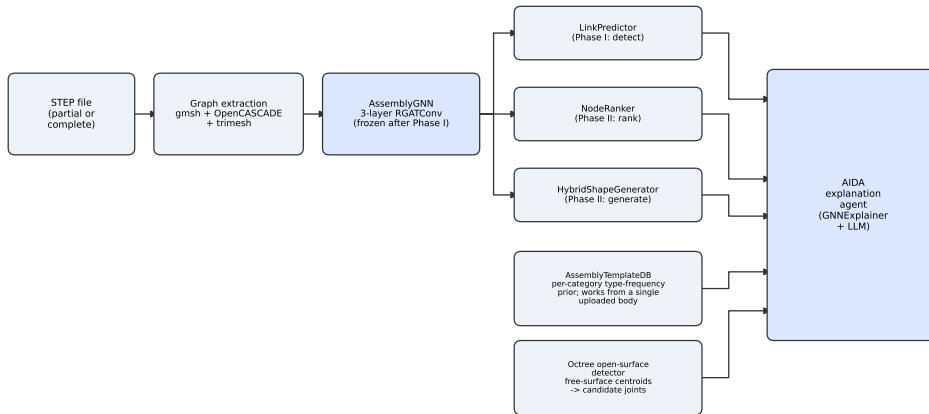


Figure 4.1: System pipeline: one shared frozen encoder feeds three task heads and a live explanation layer.

The single most consequential architectural decision is the shared, frozen encoder. The encoder is trained exactly once, end-to-end with the Phase 1 link-prediction head. It is then frozen, and every subsequent head trains only its own comparatively small parameter set on top of the fixed embeddings the encoder produces. The Phase 2 ranking head, for example, contains roughly 4,200 parameters in its projection layer against the encoder’s several hundred thousand.

This has three consequences worth stating. It keeps the marginal cost of adding a task head very low, which is what made a three-phase project feasible on a modest corpus and a single machine. It guarantees that all three phases reason over a consistent representation of the assembly. And it means a Phase 1 retrain invalidates the downstream heads, which must be retrained afterwards — a real operational constraint that is documented in the project’s own configuration notes.

4.2 Graph Construction

Each STEP assembly file is parsed using `gmsh` with the OpenCASCADE kernel, divided into individual solid bodies, and transformed into an attributed graph: one node per solid body, and one edge for each pair of bodies whose boundary face sets physically intersect after fragmentation.

The word *intersect* is doing important work in that sentence. An edge represents a genuine shared contact surface, not spatial proximity. Two bodies that sit adjacent to one another without touching receive no edge regardless of how close they are, because no shared boundary exists after fragmentation. This is a deliberate departure from the distance-threshold approach that is common in point-cloud work, and it matters: a bolt passing *near* a hole is a different mechanical situation from a bolt seated *in* it, and a proximity threshold cannot tell those apart.

4.3 Feature Engineering

4.3.1 Node Features (34 dimensions)

Every body carries a 34-dimensional feature vector, composed as follows.

Two of these groups deserve comment because they were not present in early revisions and were added in response to observed failures.

The **component-type assignment** originally relied on a single Shape-Diameter-Function rule. That proved unreliable, and it was replaced by multi-signal geometric voting that combines hole detection, bolt-head detection and absolute-size gating. The change was driven by observing specific misclassifications rather than by a metric.

The **hole-geometry block** of thirteen dimensions was added for the current corpus revision. Its most interesting member is the final flag, which records whether a hole’s local surroundings are curved rather than flat. The reasoning is mechanical rather than statistical: fasteners essentially never mount on a curved outer surface, so this single bit

Table 4.1: Node feature vector composition (34 dimensions).

Dims	Group	Description
8	Component type (one-hot)	long shaft, short shaft, thick plate, thin plate, bolt, washer, nut, body — assigned by multi-signal geometric voting
2	Scale	Log-scaled volume and exact surface area, computed with <code>trimesh</code> rather than from a bounding-box approximation
3	Bounding box	Scale-normalised extents along the three principal axes
5	Shape descriptors	Affine-invariant: elongation, flatness, two aspect ratios, sphericity
2	Shape Diameter Function	SDF mean and variance from inward ray-casting
1	Ratio	Surface-area-to-volume ratio
13	Hole geometry	Hole count, has-holes flag, through / counterbore / indentation / filled-hole fractions and their binary flags, mean and maximum hole diameter, and a curved-surroundings flag

distinguishes a genuine mounting hole from an incidental void — a distinction that no amount of size or count information conveys on its own.

4.3.2 Edge Features (6 dimensions)

Two dimensions encode the mate type and a real contact-area weight — the shared-surface area ratio between the two bodies, which replaced an earlier hardcoded constant. The remaining four encode a one-hot joint-type classification (rigid, revolute, slider, cylindrical) inferred from the local contact geometry.

The contact-area weight is worth singling out. In earlier revisions this was a fixed value, meaning every edge in the graph claimed identical contact regardless of whether two bodies shared a broad clamped face or touched along a thin line. Replacing it with a measured ratio gave the attention mechanism a genuine physical signal to weight against.

4.4 Shared Encoder: AssemblyGNN

All task heads read from one shared encoder, *AssemblyGNN*: a three-layer relational graph attention network built on RGATConv [9], with eight attention heads at the first two layers narrowing to one at the third, a hidden width of 128, and an output embedding width of 64.

Two conditioning mechanisms distinguish this encoder from a standard GAT stack.

Node-type conditioning. Both the input and output projections are node-type-

conditioned: a separate linear layer exists per component-type class, dispatched by the node’s own one-hot type. A bolt and a housing therefore pass through different learned projections. The motivation is that the same numeric feature means different things for different component classes — a high surface-area-to-volume ratio is unremarkable for a thin plate and highly informative for a body.

Joint-type conditioning. Relational attention at every layer is conditioned on the discrete joint-type argmax of each edge. A revolute joint and a rigid joint therefore pass different learned attention weights even between structurally identical node pairs. Continuous edge features — the contact-area weight and the mate-type scalar — are injected only at the first layer, to avoid repeatedly re-injecting the same signal at every depth.

4.5 Phase 1: Missing-Component Detection

Phase 1 frames missing-component detection as inductive link prediction. For a candidate body pair (u, v) , given the encoder’s node embeddings $z_u, z_v \in \mathbb{R}^{64}$, a two-layer MLP head scores the pair:

$$\text{score}(u, v) = \text{MLP}(z_u \parallel z_v \parallel \|p_u - p_v\|_2) \quad (4.1)$$

where p_u, p_v are the two bodies’ scale-normalised bounding-box centroids and $\parallel \cdot \parallel$ denotes concatenation.

The Euclidean-distance term was a deliberate late addition, and the reason it was needed is instructive. Every node feature in the vector is affine-invariant, and message passing only ever traverses *real* edges. A candidate non-edge pair being scored therefore had no signal whatsoever for the proposition “these two bodies are nowhere near each other”. A structurally implausible pair and a plausible-but-missing one were indistinguishable to the head. Adding a single scalar distance term supplied the missing spatial context.

Training uses `RandomLinkSplit` per graph, holding out 10% of edges for validation and 10% for test, disjoint from the message-passing edges. A 1:1 hard-negative-augmented positive-to-negative ratio is used in preference to the 1:2 skew that is common in the field, for reasons discussed in Section 5.5. The objective is standard binary cross-entropy.

At inference time a missing component in a real, possibly incomplete assembly is operationally identified as a body whose best candidate connection scores anomalously low relative to its structurally similar peers. This signal is surfaced to the user alongside the octree detector’s independent geometric evidence, so that two mechanisms of different

kinds must broadly agree before a strong claim is presented.

4.6 Phase 2: Next-Component Recommendation

Phase 2 answers the question of *what* component type is most likely absent. Using the embeddings of the assembly’s known nodes $\{z_i\}_{i=1}^N$, a mean-pooled context vector $c = \frac{1}{N} \sum_i z_i$ is projected through a single learned linear layer and compared to each of the eight candidate component-type embeddings by cosine similarity, scaled by a learnable CLIP-style temperature τ :

$$\text{score}(t) = \cos(W_c c, z_t) \cdot e^\tau, \quad t \in \{1, \dots, 8\} \quad (4.2)$$

The temperature is initialised at zero, so that $e^\tau = 1$ and the comparison begins as plain unscaled cosine similarity, sharpening only once training signals that it should. This was a considered choice rather than a default. The ranking head is deliberately thin — roughly 4,200 parameters in the projection alone — and unscaled cosine similarity, bounded in $[-1, 1]$, produces score differences too small for the ranking loss’s gradient to act on efficiently at that head size.

Training uses a leave-one-node-out protocol. For each graph and each held-out node, the remaining nodes form the context, and the held-out node’s true type is the positive candidate against the other seven as implicit negatives. The objective is Bayesian Personalised Ranking loss [14].

4.7 Phase 3: Hybrid Shape Generation

After recommending a component type, the system must produce an actual mesh rather than a label. *HybridShapeGenerator* attempts two strategies in a deliberate order.

4.7.1 Retrieval

A part bank built from every body in the training corpus — indexed by component type and a normalised bounding-box/scale signature — is queried for the closest match to the target region’s geometry. If the best match’s fit score clears a type-dependent threshold, the retrieved mesh is returned directly at zero additional generation cost.

The threshold is $\tau_{\text{retrieval}} = 0.6$ in general, relaxed to $\tau_{\text{retrieval}} = 0.4$ specifically for fasteners — bolts, nuts and washers. The asymmetry is intentional. The bank already holds

hundreds of real examples of these highly standardised shapes, and for such components a slightly imperfect real part reliably beats a generative hallucination.

4.7.2 Conditional VAE Fallback

When no retrieval match clears the threshold, a small conditional variational autoencoder generates a novel shape. The target region is voxelised to a 32^3 occupancy grid. The encoder comprises four 3D-convolutional blocks ($1 \rightarrow 32 \rightarrow 64 \rightarrow 128 \rightarrow 256$ channels, stride-2 downsampling, batch normalisation, LeakyReLU) producing a 128-dimensional latent distribution. The decoder mirrors this with transposed convolutions, conditioned throughout on a 77-dimensional vector: the frozen encoder’s 64-dimensional context, an 8-dimensional target-type one-hot, a 3-dimensional target bounding box, and a 2-dimensional neighbour-scale signal.

Training minimises a weighted combination of voxel-wise binary cross-entropy, a Dice-coefficient term, and a KL-divergence regulariser on the latent distribution:

$$\mathcal{L} = \mathcal{L}_{\text{BCE}} + \lambda_{\text{dice}} \mathcal{L}_{\text{Dice}} + \beta_{\text{KL}} \mathcal{L}_{\text{KL}}, \quad \lambda_{\text{dice}} = 0.5, \beta_{\text{KL}} = 0.05 \quad (4.3)$$

The Dice term exists to counteract the severe foreground/background imbalance of a sparse occupancy grid. In a 32^3 grid containing a bolt, the overwhelming majority of voxels are empty, and binary cross-entropy alone will happily converge to predicting emptiness everywhere.

4.8 Supporting Modules

4.8.1 Octree Open-Surface Detector

To identify *where* a missing component likely attaches without relying on a proprietary CAD-kernel API at inference time, an octree spatial partition — adapting the octree concept from prior mesh-watermarking work [15] — clusters the free-surface (unmated) face centroids of each body after the same fragmentation step used for graph construction. Dense clusters of free-surface area flag candidate open joints directly from raw mesh geometry.

The value of this module is that its evidence is entirely independent of the link-prediction signal. It knows nothing about the graph, the encoder or the learned embeddings; it looks only at which faces are unmated and where they cluster. When it and Phase 1 agree, the combined claim is considerably stronger than either alone.

4.8.2 AssemblyTemplateDB

A per-category component-type frequency prior, learned directly from the training corpus’s true type distributions rather than from any graph-structured model. Because it requires no graph context at all, it remains useful in the degenerate case of a single uploaded body with no known neighbours — precisely the case where Phase 1 and the ranking head have no assembly structure to reason over and therefore nothing to say.

4.8.3 Explanation Layer

GNNExplainer [12] produces post-hoc edge- and feature-attribution scores over the frozen encoder and its task heads for a given prediction. A Gemini-based agent, referred to in the application as AIDA, consumes these attributions together with the AssemblyTemplateDB prior and the octree detector’s findings, and renders them as an engineering-language explanation — identifying a specific detected assembly as an incomplete fastening operation, for example, and naming which joints require completion — rather than exposing raw attribution weights to the end user.

The agent’s persona and domain skill areas are defined declaratively in a YAML profile rather than in code, so the explanation vocabulary can be revised without touching the inference path.

This runs live against uploaded STEP files through a Streamlit front end and a FastAPI backend, with the full detect → rank → generate → explain pipeline completing in approximately 24 seconds end-to-end for a typical four-body assembly.

4.9 Algorithms

Three algorithms carry most of the system’s behaviour. They are given here in pseudocode; the corresponding source modules are listed in Table 5.2.

The critical line is the conditional on shared boundary faces. Substituting a distance threshold there would admit every pair of bodies that merely sit close together, and the resulting graph would encode spatial layout rather than mechanical connectivity.

4.10 Data Flow

Data passes through five stages, and the boundary between stages three and four is where the architecture’s central decision becomes visible.

Algorithm 1 STEP file to attributed assembly graph**Require:** STEP file F , parse timeout T **Ensure:** Attributed graph $G = (V, E, X, A)$, or **null** on failure

```

1:  $B \leftarrow \text{FRAGMENTSOLIDS}(F)$  ▷ gmsh / OpenCASCADE, bounded by  $T$ 
2: if  $B = \emptyset$  or timeout exceeded then
3:   return null ▷ reported, not silently substituted
4: end if
5:  $V \leftarrow \{v_b : b \in B\}$ 
6: for all  $b \in B$  do
7:    $M_b \leftarrow \text{BUILDMESH}(b)$ 
8:    $s_b \leftarrow \text{SHAPE SIGNALS}(M_b)$  ▷ volume, area, bbox, SDF, holes
9:    $\tau_b \leftarrow \text{CLASSIFYTYPE}(s_b)$  ▷ multi-signal geometric voting
10:   $X[v_b] \leftarrow \text{ASSEMBLENODEVECTOR}(\tau_b, s_b)$  ▷ 34-dim
11: end for
12: for all unordered pairs  $(u, v) \in V \times V$  do
13:    $S \leftarrow \text{SHARED BOUNDARY FACES}(u, v)$ 
14:   if  $S \neq \emptyset$  then ▷ genuine contact, not proximity
15:      $w \leftarrow \text{CONTACT AREA WEIGHT}(S, \text{area}(u), \text{area}(v))$ 
16:      $j \leftarrow \text{CLASSIFYJOINT}(S)$  ▷ rigid / revolute / slider / cylindrical
17:      $E \leftarrow E \cup \{(u, v)\}; \quad A[(u, v)] \leftarrow [\text{mate}, w, \text{ONEHOT}(j)]$ 
18:   end if
19: end for
20: return  $(V, E, X, A)$ 

```

1. **Ingest.** A STEP file arrives, is fragmented into solid bodies, and each body is meshed. This stage is by far the slowest and is the only one that can fail outright.
2. **Featurise.** Geometric signals are computed per body and per contact, producing the 34-dimensional node vectors and 6-dimensional edge vectors. Component types are assigned here by geometric voting.
3. **Encode.** The graph passes through the frozen encoder *once*, yielding a 64-dimensional embedding per node.
4. **Branch.** All three task heads read those same embeddings independently. No head re-encodes, and no head can influence another’s input.
5. **Explain.** Attributions, the template prior and the octree findings are bundled and rendered into engineering language.

Stage four is what makes the marginal cost of a task head small, and it is also what creates the operational constraint noted in Section 4.4: any change to stage three invalidates every head downstream of it.

Algorithm 2 Missing-component detection at inference

Require: Graph G , trained encoder f_θ , link head g_ϕ **Ensure:** Ranked candidate missing connections

- 1: $Z \leftarrow f_\theta(X, E, A)$ $\triangleright$ one forward pass, shared by all heads
 - 2: $C \leftarrow \{(u, v) : u, v \in V, u \neq v, (u, v) \notin E\}$
 - 3: **for all** $(u, v) \in C$ **do**
 - 4: $d \leftarrow \|p_u - p_v\|_2$ $\triangleright$ scale-normalised centroids
 - 5: $\sigma_{uv} \leftarrow \text{sigmoid}(g_\phi(z_u \| z_v \| d))$
 - 6: **end for**
 - 7: $O \leftarrow \text{OCTREEOPENSURFACES}(G)$ $\triangleright$ independent geometric evidence
 - 8: **return** $\text{SORTDESCENDING}(C \text{ by } \sigma)$, corroborated against O
-

Algorithm 3 Hybrid shape generation

Require: Component type t , target bounding box β , context vector c **Ensure:** Mesh M and its provenance

- 1: $\tau \leftarrow 0.4$ **if** $t \in \{\text{bolt, nut, washer}\}$ **else** 0.6
 - 2: $(M_r, \text{fit}) \leftarrow \text{QUERYPARTBANK}(t, \beta)$
 - 3: **if** $\text{fit} \geq \tau$ **then**
 - 4: **return** $(M_r, \text{RETRIEVAL})$ $\triangleright$ a real part, at zero generation cost
 - 5: **end if**
 - 6: $z \sim \mathcal{N}(0, I)$
 - 7: $\kappa \leftarrow \lceil c \| \text{ONEHOT}(t) \| \beta \| \text{NEIGHBOURSCALE} \rceil$ $\triangleright$ 77-dim
 - 8: $V \leftarrow \text{sigmoid}(\text{DECODE}(z, \kappa))$ $\triangleright$ 32^3 occupancy
 - 9: **return** $(\text{MARCHING CUBES}(V), \text{GENERATED})$
-

Chapter 5

Implementation Details

5.1 Development Environment

All three phases are implemented in Python 3.12 using PyTorch 2.12 with PyTorch Geometric [13], and trained on a single Apple-Silicon GPU through the Metal Performance Shaders backend. Geometry processing uses `gmsh` with the OpenCASCADE kernel for STEP parsing and fragmentation, and `trimesh` for mesh-level measurements.

All hyperparameters and data paths live in a single `config.yaml` rather than being distributed through the source. This was not merely tidy practice: across roughly forty training revisions it was the only reliable way to know precisely which configuration produced a given checkpoint, and the file accumulated inline commentary explaining why particular values were chosen, which became the project’s most useful engineering record.

5.2 Corpus Construction

The training corpus consists of 749 STEP-format assembly models, curated evenly across seven mechanical-part categories relevant to industrial fastening and flow-control mechanisms — bench vices, C-clamps, pipe vices, gate valves, press tools, tool posts and crane hooks — at 107 models per category.

After parsing through the `gmsh`/OpenCASCADE pipeline, 648 of the 749 models (86.5%) yielded usable graphs. The remainder failed under a bounded-timeout parsing policy. Those failures were not uniformly distributed: they concentrated heavily in the gate-valve category, which returned a 51.4% yield against 82–99% for the other six. Gate valves also have this corpus’s largest mean graph size at 93.9 bodies per assembly against an overall mean of 38.8, which points to geometric complexity as the likely common cause

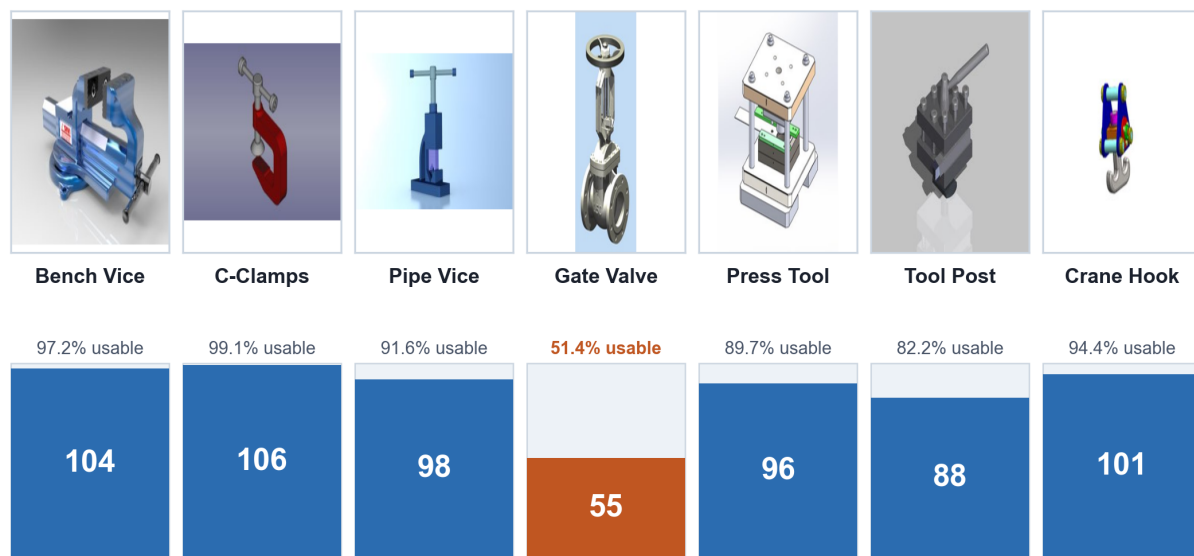
rather than anything specific to the category’s file provenance.

The resulting 648 graphs contain 25,155 labelled bodies and 46,767 undirected contact edges in total. Table 5.1 gives the per-category composition.

Table 5.1: Corpus composition by category (648 usable graphs from 749 models).

Category	Graphs	Yield	Mean bodies/graph	Dominant type
C-Clamps	106	99.1%	9.6	body
Bench vice	104	97.2%	30.6	body
Crane hook	101	94.4%	78.5	body
Pipe vice	98	91.6%	15.8	body
Press tool	96	89.7%	32.8	body
Tool post	88	82.2%	35.9	bolt
Gate valve	55	51.4%	93.9	body
Total	648	86.5%	38.8	—

Figure 5.1 shows the same composition visually: one reference image of each category, and beneath it the number of usable graphs that category produced from its 107 models, in the same order. Gate valves are the clear outlier at 55 usable graphs (51.4%); the other six categories each yield at least 82%.



Bar = usable assembly graphs; full track = the 107 models in each category. Total: 648 of 749 models (86.5%).

Figure 5.1: Corpus overview. Top: a reference image of each of the seven categories. Bottom: usable assembly graphs obtained from the 107 models in each category (648 of 749 in total, 86.5%); gate valves, highlighted, have the lowest yield at 51.4%.

An earlier iteration of this project worked from a smaller, scraped corpus of 192 models. Replacing it with a deliberately curated and category-balanced 749-model corpus

was the single most expensive undertaking of the project in wall-clock terms, and also the one that produced the largest measured improvement — a point returned to in Section 6.2.

5.3 Module Implementation

Table 5.2: Principal source modules.

Module	Responsibility
<code>dataset.py</code>	STEP $\rightarrow$ graph pipeline: fragmentation, node/edge feature extraction, component-type voting, caching, cross-validation splits
<code>model.py</code>	AssemblyGNN encoder, LinkPredictor , NodeRanker , and the type-conditioned linear layer
<code>train.py</code>	Phase 1 training: five-fold CV, early stopping, edge-budgeted batching, checkpoint promotion gating
<code>train_ranker.py</code>	Phase 2 training: leave-one-node-out sampling, BPR loss, frozen-encoder reuse
<code>train_shape_gen.py</code>	Phase 3 training: conditional VAE over voxel grids
<code>part_bank.py</code>	Retrieval index over training-corpus geometry, keyed by type and scale signature
<code>shape_generator.py</code>	HybridShapeGenerator : retrieval-first with VAE fallback
<code>slot_detector.py</code>	Locates unfilled hole seats on host bodies and infers missing members from hole-pattern symmetry
<code>rebuild.py</code>	Reconstructs missing components: copies surviving instances, retrieves shafts, springs and knob, exports <code>.glb</code>
<code>surface_analyzer.py</code>	Octree open-surface detector
<code>assembly_templates.py</code>	AssemblyTemplateDB component-type frequency prior
<code>explainer.py</code>	GNNE explainer integration over the frozen encoder
<code>skills_agent.py</code>	Gemini-based AIDA explanation agent, profile-driven
<code>evaluate.py</code>	AUC, AP, Hit@K, MRR, NDCG@K, per-class breakdowns
<code>api.py</code>	FastAPI inference service
<code>app.py</code>	Streamlit dual-panel user interface

5.4 Training Protocol

Table 5.3 lists the training configuration for each stage.

Phase 1 batches graphs by a cumulative edge-count budget rather than a fixed graph count. The reason is specific to relational attention: its per-edge relation-weight computation scales with a batch’s *total edge count*, not its node count. A few of the corpus’s larger assemblies exceed 1,000 edges each, so an unbudgeted batch that happened to combine several of them could exceed available GPU memory even though every individual graph fitted comfortably alone. Section 5.6 describes how this was discovered.

Table 5.3: Training configuration by stage.

Parameter	Phase 1 (encoder)	Phase 2/3 (heads)
Optimiser	Adam	Adam
Learning rate	1×10^{-3} , ReduceLROnPlateau (factor 0.5)	1×10^{-3} , fixed
Weight decay	5×10^{-4}	1×10^{-5} (ranker)
Batch composition	Edge-budgeted, cap 300 edges/batch	Full-graph context per sample
Early stopping	20 epochs (LR patience 8)	None — fixed 30 / 60 epochs
Dropout	0.3	—
Cross-validation	True 5-fold, per-fold reseeded splits	5-fold protocol, fold 0 reported
Max epochs	200	30 (ranker) / 60 (shape gen)

5.5 Evaluation Metrics

Phase 1 is evaluated under true five-fold cross-validation — five independent fold assignments, not a single fixed split reused five times — with mean AUC-ROC and mean average precision as primary metrics. Chance-level AP is reported alongside AP as a reference, because AP is sensitive to class imbalance in a way AUC-ROC is not.

That sensitivity was not a theoretical concern. Earlier revisions used a negative-sampling ratio of 1:0.5, meaning positives outnumbered negatives two to one in every evaluation set. Since a random scorer’s expected AP equals positive prevalence, any below-1.0 ratio inflates AP for *any* model, including a poor one, entirely independently of real ranking skill. Moving to the standard 1:1 convention made the AP figures comparable to published baselines, and chance-level AP is now reported alongside AP so that the skew can never again be hidden silently.

Phase 2 is evaluated with Hit@1, Hit@5, mean reciprocal rank and NDCG@5 against a corpus-wide majority-class baseline. The baseline is computed from true corpus-wide type frequency rather than from the sample pool, which is the more conservative of the two definitions.

Phase 3 is evaluated with mean intersection-over-union and Chamfer distance between generated and ground-truth occupancy grids on a held-out test split, with the best checkpoint selected by validation IoU and Chamfer rather than by training loss.

5.6 Engineering Challenges

Building a pipeline that runs end-to-end on real, imperfect STEP geometry surfaced several infrastructure-level problems with no direct analogue in a purely algorithmic de-

scription of the system. The ones reported here had the largest measured impact, and several materially affected the numbers in Chapter 6.

5.6.1 Batch-Composition Out-of-Memory

Relational attention’s memory cost scales with a batch’s total edge count. An early fixed-graph-count batching scheme worked until training reached the corpus’s larger assemblies, at which point a single unlucky batch composition triggered an out-of-memory crash requiring roughly 23 GB against a 20.13 GB GPU ceiling.

Capping batches by cumulative edge count resolved this, but only after two intermediate failures that are worth recording. A cap of 3,000 edges still crashed, because backpropagation’s retained-activation memory runs well above the forward-pass cost the figure had been sized against. A further reduction to 500 edges still crashed intermittently — and the intermittency turned out to be the interesting part. The batch sampler’s shuffle seed was not tied to the resumed epoch number, so an auto-restarted run deterministically replayed the same crash-triggering batch composition rather than varying it. The final cap of 300 edges per batch, combined with fixing the seed-reset bug, eliminated the failure mode entirely.

5.6.2 Silent Process Termination

Training processes were observed terminating silently mid-fold under host memory pressure, with no exception raised and no entry in the training log — only a stalled process. A watchdog script was built that monitors for output staleness and GPU cache growth, and automatically resumes training from the last-saved checkpoint at the correct fold and epoch rather than restarting the entire run. Given that a full five-fold run occupies several days of wall-clock time, whole-run restarts were not affordable.

5.6.3 A Promotion-Gate Comparison Bug

The gate deciding whether a newly trained checkpoint should replace the serving model compared candidate metrics as a lexicographic tuple. Because Python’s tuple comparison short-circuits on the first unequal element, this silently ignored average precision whenever two candidates tied on AUC-ROC — allowing a regression on the second metric to be promoted undetected. The gate now requires both metrics to improve, or to remain within a defined tolerance, before promotion.

This bug is notable less for its severity than for its invisibility. It produced no error, no warning and no anomalous number; it simply made one of two reported metrics

decorative.

5.6.4 Component-Classification Errors

Two geometry-classification bugs were found and fixed during development. A fastener-orientation error aligned a bolt's *short* bounding-box axis onto its mating joint instead of its long shaft axis; this was corrected by using the bounding box's middle extent to distinguish rods from disks. A whole-body hole-region heuristic collapsed twelve real, physically distinct bolt holes on a single plate into one merged detection, and replacing it with per-hole cylindrical-face detection recovered all twelve as separate candidates.

5.6.5 Checkpoint Write Failures and Split Reproducibility

Two further issues were resolved. Checkpoint writes intermittently failed with an operating-system permission error traced to cloud-drive file locking on the output directory. Separately, cross-validation splits were found not to be exactly reproducible, because `get_splits()` seeded only one of the two random number generators in play; seeding both per fold and per graph now allows the same checkpoint to be re-evaluated to the same AUC, which was verified directly.

Chapter 6

Results and Discussion

Table 6.1 gives an overview of all three phases against their design targets before the detailed discussion below.

Table 6.1: Summary of results against design targets (749-model corpus, true five-fold cross-validation).

Stage	Metric	Target	Achieved	Met
Phase 1 (detect)	Mean AUC-ROC / AP	≥ 0.85 / ≥ 0.82	0.9206 / 0.9020	Yes
Phase 2 (rank)	Hit@5 / MRR	≥ 0.70 / ≥ 0.60	0.9101 / 0.6102	Yes
Phase 3 (generate)	IoU / Chamfer	≥ 0.35 / ≤ 0.08	0.6459 / 0.0377	Yes

6.1 Phase 1: Missing-Component Detection

Table 6.2 reports Phase 1’s five-fold cross-validated performance. Mean AUC-ROC is 0.9206 (± 0.0121) and mean AP is 0.9020 (± 0.0120) against a chance-level AP of 0.500.

Table 6.2: Phase 1 link-prediction results, true five-fold cross-validation.

Fold	AUC-ROC	AP	Chance AP
1 (best)	0.9370	0.9157	0.500
2	0.9050	0.8881	0.500
3	0.9138	0.8919	0.500
4	0.9217	0.9025	0.500
5	0.9256	0.9116	0.500
Mean $\pm$ std	0.9206 $\pm$ 0.0121	0.9020 $\pm$ 0.0120	0.500

The low standard deviation across folds matters as much as the mean. A cross-validated spread of ± 0.0121 indicates the encoder separates real from candidate connections consistently, rather than owing its headline number to one favourable split.

An earlier experiment in this project made that distinction concrete: moving from a fixed reused test set to true five-fold assignment left the mean essentially unchanged but roughly *tripled* the observed variance, revealing that the fixed test set had been masking genuine fold-to-fold spread rather than inflating the average.

6.2 Development History

The Phase 1 result was not reached in a single pass. Thirty-nine tracked training revisions preceded it. Figure 6.1 plots the full history alongside the final run’s convergence.

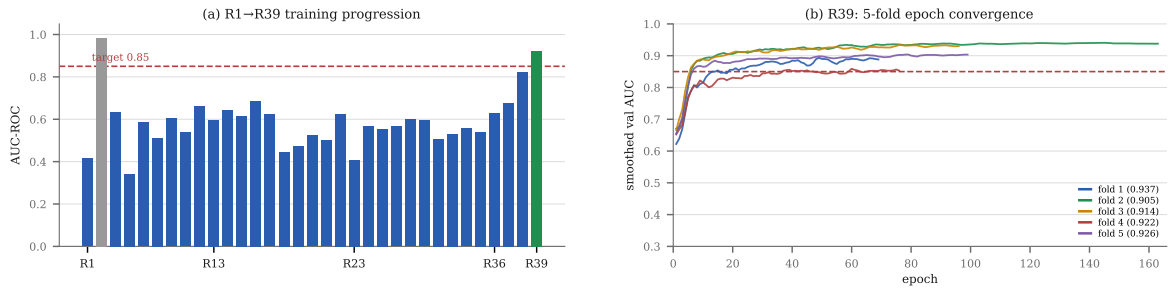


Figure 6.1: (a) Mean and best AUC across all 39 tracked training runs, with R39 (749-model corpus) highlighted. (b) R39’s five-fold validation-AUC convergence, with final test AUC per fold in the legend.

Table 6.3 traces the three consecutive revisions on an earlier 238-model corpus that produced the largest single-run gains, because which changes mattered is more instructive than the final number alone.

Table 6.3: Three consecutive development runs producing the largest single-run gains (238-model corpus revision; superseded by the 749-model results in Table 6.2).

Run	Change	Mean AUC
R35	Populated previously dead hole-count / joint-type features	0.539
R36	+ spatial link signal, real contact-area edge weight	0.628 (+0.089)
R37	+ negative-ratio 0.5 $\rightarrow$ 1.0 fix, promotion-gate fix, seeded splits	0.677 (+0.049)

The single largest jump — R35 to R36, at +0.089 mean AUC — followed directly from discovering that two node and edge features, hole count and joint type, had been silently zero-valued since the project’s inception. They were being sourced from a file path that had never existed. Once populated with real values, and paired with the spatial link signal from Equation 4.1 and a genuine rather than hardcoded contact-area edge weight, the encoder gained access to information it had never actually had.

The second-largest gain, R36 to R37 at +0.049, came from correcting the negative-sampling ratio described in Section 5.5, alongside fixing the promotion-gate bug.

The pattern across both is worth stating plainly, because it runs against expectation: **neither gain came from architecture search or hyperparameter tuning**. Both came from correcting silently broken inputs to metrics. The largest remaining improvement, from roughly 0.68 to 0.92, came from replacing the corpus itself — a data intervention, not a modelling one.

6.3 Phase 2: Next-Component Recommendation

NodeRanker achieves Hit@1 of 0.4093 against a corpus-wide majority-class baseline of 0.3442. This is a genuine, non-trivial improvement over always predicting the plurality class, and it is worth noting that earlier corpus revisions produced a head that tied the baseline exactly — a failure mode that took several attempts to move past. Hit@5 reaches 0.9101, MRR 0.6102, and NDCG@5 0.6754.

Table 6.4: Phase 2 ranking metrics against the majority-class baseline.

Metric	Validation	Test	Baseline
Hit@1	0.4188	0.4093	0.3442
Hit@5	0.8885	0.9101	—
MRR	0.6041	0.6102	—
NDCG@5	0.6621	0.6754	—

The aggregate figure conceals a real weakness, and Table 6.5 exposes it.

Table 6.5: NodeRanker per-class Hit@1 on the test split, with predicted versus true class share.

Component type	n	Hit@1	True share	Predicted share
body	222	0.689	34.3%	53.1%
nut	36	0.444	5.6%	9.7%
thick_plate	111	0.360	17.1%	14.8%
bolt	98	0.316	15.1%	12.0%
long_shaft	71	0.225	11.0%	5.6%
short_shaft	47	0.149	7.3%	2.8%
thin_plate	42	0.024	6.5%	1.1%
washer	18	0.000	2.8%	0.5%

Performance is markedly uneven. The majority class reaches 0.689 while **washer** sits at exactly zero and **thin_plate** at 0.024. The head also over-predicts **body** relative to its true frequency, at 53.1% of predictions against a 34.3% true share.

This is a majority-class bias, and it is unresolved. It is reported here rather than obscured behind the headline Hit@1 because the distinction matters for anyone considering

deploying the ranking head: the system is substantially more reliable at recommending a housing than a washer, and a user should know that. Section 9.2 returns to it.

6.4 Phase 3: Shape Generation

HybridShapeGenerator reaches a test IoU of 0.6459 and Chamfer distance of 0.0377, against targets of $\text{IoU} \geq 0.35$ and $\text{Chamfer} \leq 0.08$. The selected checkpoint recorded validation IoU 0.6640 and Chamfer 0.0359.

Table 6.6: Phase 3 shape-generation metrics.

Metric	Validation	Test	Target
IoU	0.6640	0.6459	≥ 0.35
Chamfer distance	0.0359	0.0377	≤ 0.08
Total loss	0.2122	0.2234	—
BCE term	0.0736	0.0749	—
Dice term	0.2454	0.2670	—
KL term	0.3191	0.2999	—

One result requires careful interpretation. Across all 285 test samples, *none* fell below the retrieval confidence threshold, so the VAE fallback was never invoked during this evaluation run. It would be easy to read that as evidence that the generative component is unnecessary. It is not. It indicates that retrieval alone currently covers the test distribution at the configured threshold — a result specific to this corpus’s category coverage. A corpus containing more geometrically varied or less standardised components would exercise the fallback, and the threshold analysis would need re-running.

6.5 Ablation Studies

Four controlled experiments were run to test design decisions that would otherwise rest on assumption. Each is reported with its outcome, including the two that did *not* support the change being tested.

6.5.1 Encoder Benchmark

RGATConv was benchmarked against GATv2, GraphSAGE and GIN. Per-stage widths were held constant across all four candidates, so the comparison isolates the aggregation mechanism rather than confounding it with layer capacity. GraphSAGE and GIN lose edge features entirely — neither accepts an `edge_dim` — which is a real asymmetry, and

a deliberate one: part of what the benchmark was meant to surface was whether the joint-type and contact-area edge features are actually load-bearing.

Table 6.7: Encoder benchmark. Two-fold screening against full five-fold validation.

Encoder	Edge features used	2-fold screen	Full 5-fold
RGATConv	Joint type + contact area	lower	0.6765
GraphSAGE	None	apparently higher	0.6717
GATv2	Contact area (layer 1)	—	—
GIN	None	—	—

The result is worth recording precisely because it is a near-miss. Two-fold screening suggested GraphSAGE was ahead, and acting on that screen would have removed relational attention from the architecture. Full five-fold validation showed an essential tie — 0.6765 against 0.6717, well inside the fold-to-fold spread. RGATConv was retained.

The methodological lesson generalises: an under-powered screening protocol does not merely produce a noisy estimate, it can invert the ranking of two candidates and drive a permanent architectural decision the wrong way.

6.5.2 Capacity Ablation

Two capacity parameters were varied against the then-current baseline of 0.6765 mean AUC.

Table 6.8: Capacity ablation outcomes.

Change	Mean AUC	Outcome
Baseline (dropout 0.2)	0.6765	Reference
Dropout $\rightarrow$ 0.3	0.6875	Promoted; retained in the final configuration
Hidden dim $\rightarrow$ 256	—	Hard MPS out-of-memory crash; not evaluable on this hardware

Raising dropout to 0.3 produced a modest but real gain at full five-fold rigour and was promoted. Widening the hidden dimension to 256 could not be assessed at all: it exceeds the Metal Performance Shaders allocation ceiling of the development machine. That is a hardware limit rather than a negative result, and it is recorded as such rather than as evidence that wider is not better.

6.5.3 Learning Curve

To establish whether the system was data-limited or model-limited, Phase 1 was retrained on nested subsets of the corpus.

Table 6.9: Phase 1 performance against training-corpus fraction.

Corpus fraction	Mean AUC	Trend
25%	0.5736	—
50%	intermediate	rising
75%	intermediate	rising
100%	0.6627	no plateau observed

Performance scales cleanly with corpus size across the full range, with no sign of saturation at 100%. The system was therefore *data-limited* at that point, not architecture-limited — which directly motivated the corpus expansion from 238 to 749 models described in Section 5.2, and predicted the large gain that expansion subsequently produced.

6.5.4 Cross-Validation Protocol

An early protocol used a fixed test set reused across all five folds. It was replaced with true five-fold assignment, in which each fold rotates a genuinely independent test partition.

Table 6.10: Fixed test set versus true five-fold assignment, same corpus.

Protocol	Mean AUC	Std
Fixed test set, reused $\times 5$	0.6875	± 0.0274
True five-fold assignment	0.6937	± 0.0702

The means are essentially identical; the standard deviation is roughly two and a half times larger under the honest protocol. The fixed test set had not been *inflating* the reported mean — it had been *masking* genuine fold-to-fold variance. Every Phase 1 number reported in this chapter uses the true five-fold protocol.

6.5.5 Part Bank Quality Gate

The retrieval bank was rebuilt with a geometric quality gate applied at indexing time, rejecting degenerate and malformed bodies.

Table 6.11: Part bank composition before and after the quality gate.

Bank version	Parts	Assemblies
No gate	4,500	187
With quality gate	3,843	193

The gate removed roughly 15% of indexed parts while *increasing* the number of contributing assemblies, which is the expected signature of eliminating many degenerate

fragments drawn from a few pathological files. Comparing the two rebuilds also surfaced a signed-versus-absolute volume bug that had been silently admitting inverted-normal bodies into the bank.

6.6 Findings from Exploratory Data Analysis

A full exploratory pass over the final 648-graph corpus, run *after* model training had already completed successfully, surfaced a data-quality issue invisible to the training metrics themselves.

The SDF-mean node feature is exactly zero for 25,142 of 25,155 bodies — 99.95% of the entire corpus. Only 13 bodies carry a non-zero value, and the paired SDF-variance feature moves in near-perfect lockstep with it (Pearson $r > 0.999$), which is consistent with a feature that is silently constant rather than genuinely near-zero for physically flat geometry.

This mirrors the earlier instance described in Section 6.2, where hole-count and joint-type features were found to have been dead since the project’s inception. That earlier discovery produced the single largest one-run performance gain of the entire project.

The finding is reported here unresolved, rather than omitted, because it demonstrates something worth stating: a model reaching strong aggregate metrics is not sufficient evidence that every input feature is contributing. A 34-dimensional feature vector of which one pair is constant still trained to 0.92 AUC. Targeted exploratory analysis of the corpus remains necessary even after training succeeds, and is recommended here as standard practice for any comparable pipeline.

A second, related finding emerged from the same analysis. Some 72.1% of bolts and 83.5% of washers in the corpus are flagged by the geometric hole detector as having holes — a higher rate than `thick_plate` bodies at 47.8%, which structurally host the through-holes that fasteners actually sit in. A washer’s own bore is a genuine hole, so a high rate there is expected; a solid bolt normally is not. The elevated rate plausibly reflects the detector picking up thread-groove or head-recess geometry as false positives on a meaningful fraction of bolts. This is flagged as a candidate data-quality issue for future correction rather than a confirmed bug, since it has not yet been manually verified against individual STEP files at scale.

6.7 Discussion

Three observations stand out from the results as a whole.

Data quality dominated architecture. The two largest single-run gains in a thirty-nine-revision history both came from repairing silently broken inputs, and the largest improvement overall came from replacing the corpus. No architecture search or hyperparameter sweep in this project produced a comparable effect. For a project of this scale on a modest corpus, effort spent auditing inputs returned considerably more than effort spent searching model space.

Aggregate metrics conceal structure. Phase 2’s Hit@1 of 0.4093 genuinely exceeds baseline, and simultaneously conceals a head that cannot identify a washer at all. Reporting per-class breakdowns alongside aggregates is not a courtesy; without Table 6.5 the Phase 2 result would be actively misleading.

Retrieval-first was the right default for this domain. Industrial fasteners are highly standardised, and a part bank holding hundreds of real examples will usually contain something closer to correct than a 32^3 -resolution generative sample. The architecture reflects a judgement about the domain rather than about model capability.

Chapter 7

Application Interface and Demonstration

This chapter documents the deployed application through screenshots of the running system, captured on 18 September 2026. The application is the Streamlit front end described in Section 4.8.3: a single page with a control sidebar and a dual-panel workspace, in which a user uploads a STEP file and receives the detected, ranked, generated and explained results without leaving the page. The demonstration asset is a tool-post assembly, derived from the corpus model `Tool_Post_10`, from which fasteners, springs, shafts and the knob were deliberately removed so that the correct answer is known (Section 7.4.2).

7.1 Application Overview

Figure 7.1 shows the application in its initial state, before any file is loaded. The window has three regions. The *sidebar* on the left carries the viewer settings, the trained-model information and the activity log. The *workspace* occupies the centre and is divided into an *Input* panel and a *Results* panel. The *AIDA panel* below the workspace stays empty until inference finishes, and then holds the engineering-language explanation of the predictions.

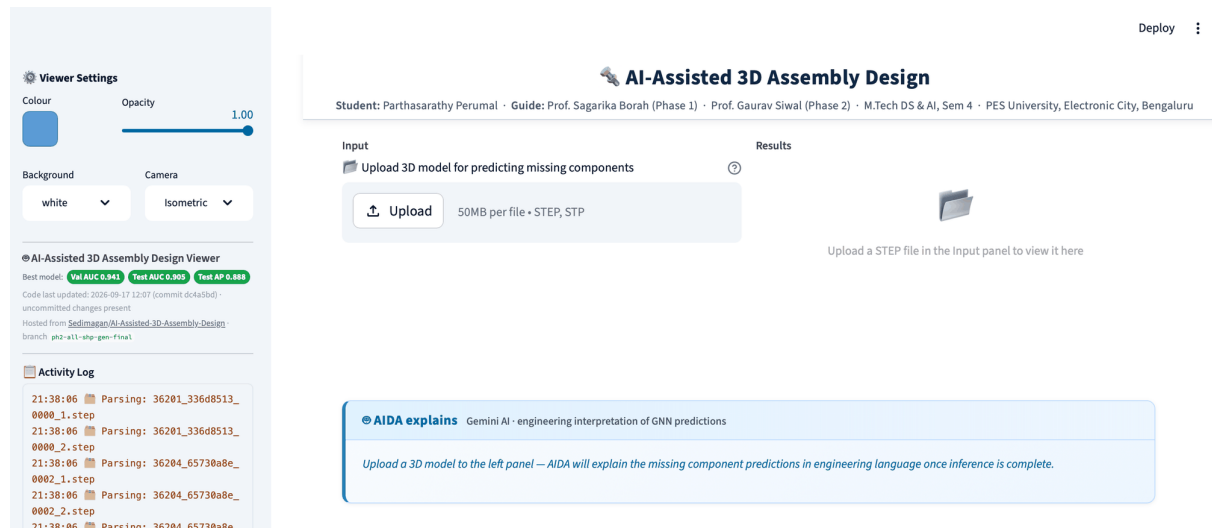


Figure 7.1: Full application window in its initial state: sidebar (left), Input and Results panels (centre) and the AIDA explanation panel (bottom).

7.2 Sidebar Controls

7.2.1 Viewer Settings

The viewer settings, shown in Figure 7.2, control how the 3D model is drawn and do not affect any prediction. *Colour* sets the base colour of the assembled parts, *Opacity* ranges from 0.10 to 1.00 so that internal components such as balls and springs can be seen through their housings, *Background* sets the scene background (white in the figure), and *Camera* selects the viewing preset (isometric in the figure).

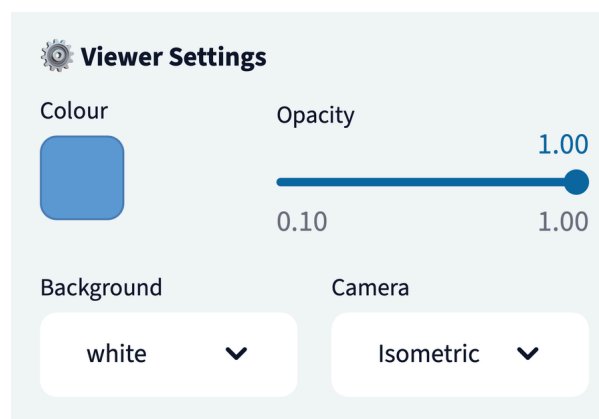


Figure 7.2: Viewer Settings: part colour, opacity, background and camera preset.

7.2.2 Trained-Model Information

The sidebar reports which model is currently serving predictions (Figure 7.3). Three badges give the validation AUC-ROC (0.941), the test AUC-ROC (0.905) and the test average precision (0.888) recorded for the promoted checkpoint; a badge is green when its AUC is at least 0.70. These are single-checkpoint figures and differ from the five-fold means of Table 6.2, which are the results this project reports. Beneath the badges the application shows the date and commit of the code it is running, whether uncommitted changes are present, and the repository and branch it was served from, so that any screenshot can be traced to a specific code state.

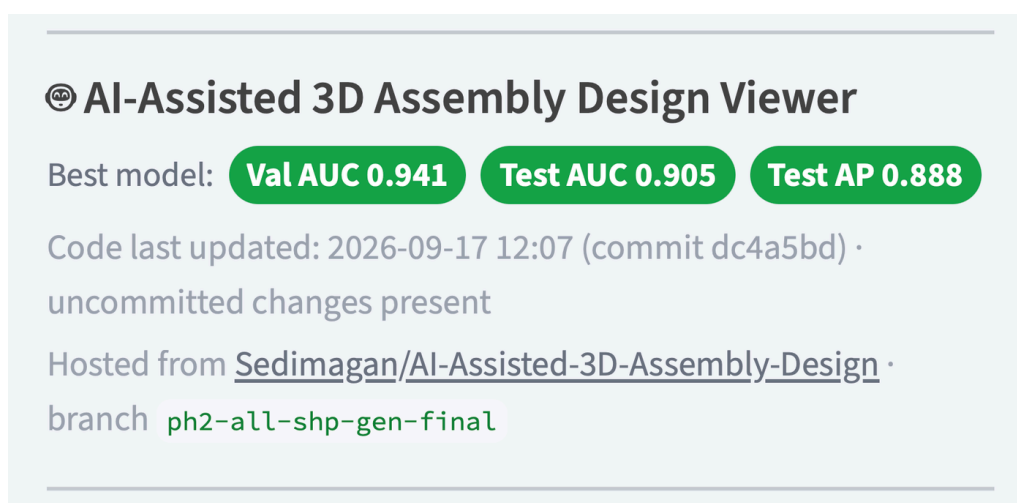


Figure 7.3: Trained-model information: metrics of the promoted serving checkpoint and the code version the application is running.

7.2.3 Activity Log

The activity log (Figure 7.4) records each pipeline stage with a timestamp as it runs: STEP files being parsed, the assembly graphs built and saved, the split summary, and the stage markers of the inference run, for example [2/4] Building model. It makes the multi-second processing time visible to the user and is the first place to look when a stage fails. A *Reset All & Log* control (Figure 7.9) clears it.

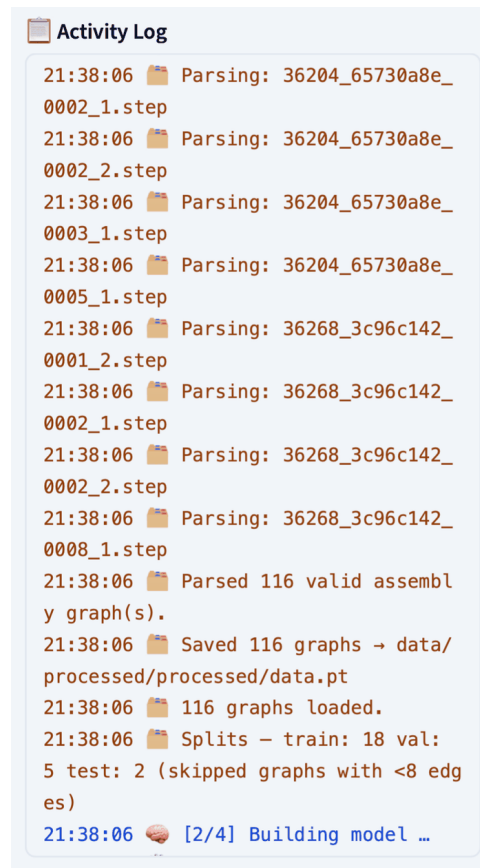


Figure 7.4: Activity Log: timestamped pipeline stages.

7.3 Input and Results Panels

7.3.1 Model Upload

The Input panel accepts a single STEP or STP file of up to 50 MB through the upload control shown in Figure 7.5(a). Until a file is loaded, the Results panel displays a prompt asking for one (Figure 7.5(b)).

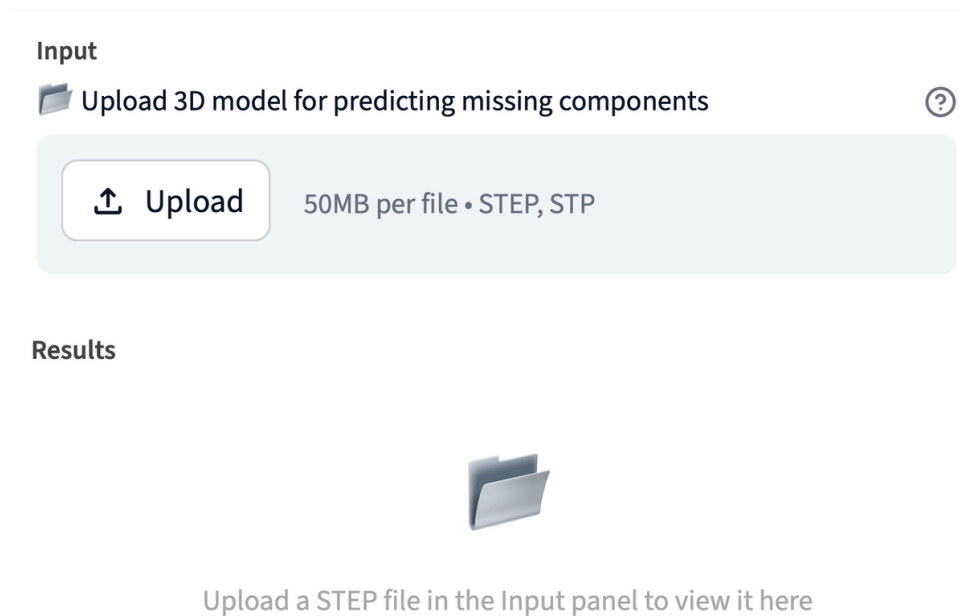


Figure 7.5: (a) Input panel with the STEP/STP upload control. (b) Results panel before a file is loaded.

7.3.2 Inspecting the Uploaded Assembly

Once a file is parsed, the Results panel shows the model in an interactive 3D viewer (Figure 7.6). A *Original/Result* toggle switches between the uploaded geometry and the prediction overlay, and *Highlight* toggles emphasis on the selected component. Each body appears as a separately switchable entry in the legend (*Part 1* to *Part 4* here), and the toolbar provides screenshot capture, zoom, pan, orbit, turntable rotation, home view, camera reset and full-screen mode. The model can be inspected from any angle before any prediction is run, which is how a user confirms that the file loaded as intended.

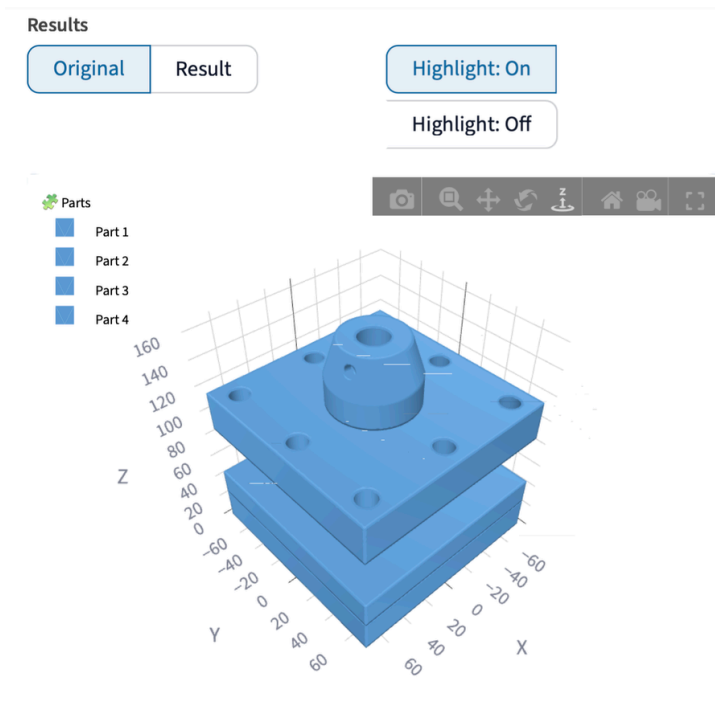


Figure 7.6: Original view of the uploaded partial tool-post assembly, before prediction.

7.4 Prediction Results

7.4.1 Result View

Selecting *Result* overlays the predictions on the uploaded model (Figure 7.7). Components are grouped in the legend by their status. *Parts* (blue) are components with their expected contacts. *Not Assembled* (orange, marked with a warning symbol) are components flagged either because they touch nothing or because they have fewer contacts than other instances of the same part. *Suggested Shapes* (pink) are the geometries proposed for the missing components. Each group can be shown or hidden individually, and the row of buttons beneath the viewer switches whole groups on or off at once. The caption line under the viewer reports the file name and the mesh size that was processed (here 76,996 points and 154,008 faces).

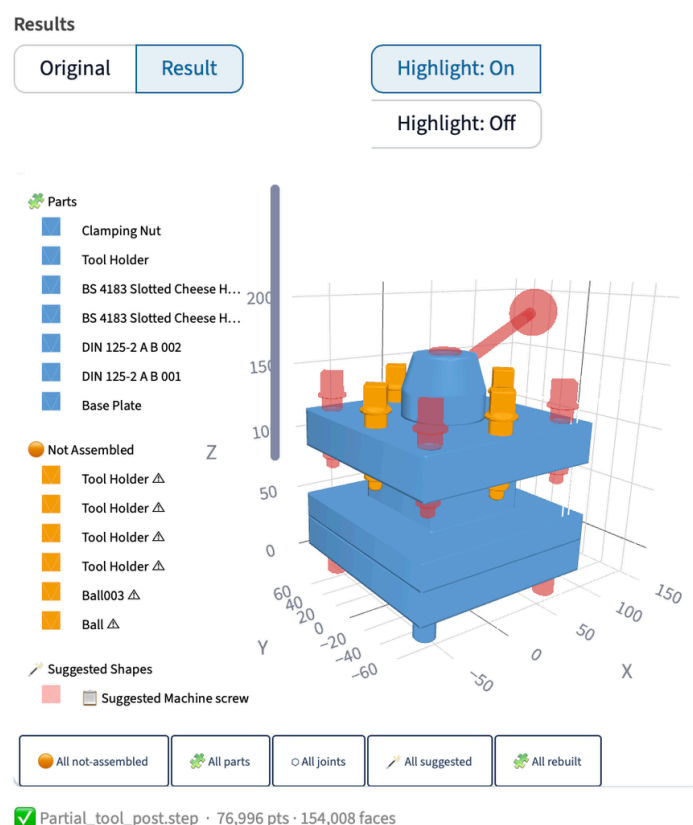


Figure 7.7: Result view of the partial tool post: Parts (blue), Not Assembled (orange) and the Suggested Shapes group (pink).

The same components appear again in the *Reconstructed* group, coloured by category, in Figure 7.8; how that group is produced is described in Section 7.4.2.

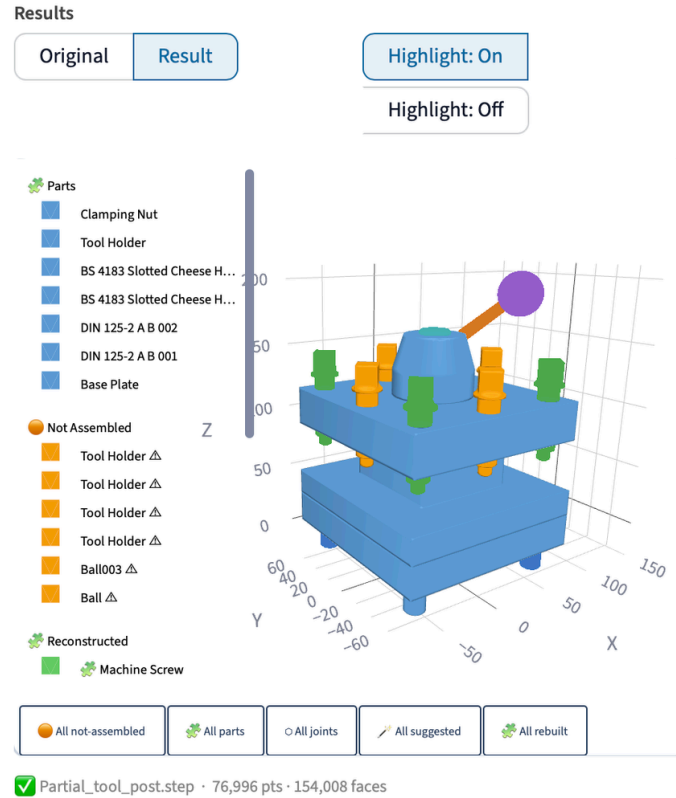


Figure 7.8: The same components in the Reconstructed group, coloured by component category.

7.4.2 Reconstructed Components

For tool-post assemblies the application additionally reconstructs the missing components and draws them in a separate *Reconstructed* group, coloured by component category so that fasteners, springs, shafts and the knob can be told apart (Figure 7.8). The step is implemented in `slot_detector.py` and `rebuild.py`, and works in three stages. First, it locates *open seats*: cylindrical holes on the host bodies with nothing seated in them, plus positions implied by the rotational symmetry of a hole pattern that has surviving members. Second, it assigns each seat a component family from the seat geometry and the parts around it. Third, it obtains the geometry. Where an identical part survives elsewhere in the upload, its mesh is copied and translated to the empty seat. Otherwise the part is retrieved from the part bank or from a small library of springs, which the bank’s watertightness gate cannot hold, using the corpus assemblies most similar to the upload to choose between candidates. The components in Figure 7.8, shown above, are in the same positions as the suggested shapes of Figure 7.7, drawn under the category colouring instead.

The reconstruction was scored against the withheld original. Table 7.1 reports, for each component family, the distance between the rebuilt part’s centroid and the matching

part in the original assembly (position), and the mean surface distance between the two meshes after centring both (shape and orientation). Across the 17 rebuilt components the mean position error is 0.27 mm and the mean shape error is 0.39 mm. Re-running with the original model’s own parts excluded from retrieval gives the same figures. Two independent tessellations of the *same* solid differ by roughly 0.2–0.6 mm, so shape errors inside that band are indistinguishable from an exact match. The central shaft (0.83 mm) is the only family above it, and the knob has the largest position error (2.93 mm).

Table 7.1: Reconstruction accuracy against the original Tool_Post_10 (17 components; mean per family, in mm).

Family	n	Geometry source	Position	Shape
Machine screw	4	Copied from surviving instance	0.00	0.41
BS 4183 screw	2	Copied from surviving instance	0.00	0.49
DIN washer	2	Copied from surviving instance	0.00	0.29
Ball	2	Copied from surviving instance	0.00	0.10
Compression spring	4	Retrieved (spring library)	0.07	0.36
Central shaft	1	Retrieved (part bank)	0.32	0.83
Side shaft	1	Retrieved (part bank)	1.14	0.45
Knob	1	Retrieved (part bank)	2.93	0.54
All	17		0.27	0.39

This result should be read narrowly. It is one assembly. The family rules and the mount-hole rule for the side shaft are calibrated for the tool-post category, and that same model was used to tune them, so the table demonstrates that the mechanism can recover a known answer rather than establishing how it generalises to unseen assemblies or other categories. The exclusion test removes the original’s parts from retrieval but does not remove the model from the calibration. A held-out evaluation on further partial tool posts, and an equivalent calibration for the other six categories, remain to be done.

7.4.3 Prediction Details Panel

When inference completes, a green banner reports how many missing components were reconstructed and placed (Figure 7.9). The reconstructed components can be expanded into a table giving each component’s position, how its seat was found, where its shape came from, and the part used, and the result can be downloaded as a `.glb` file for use in other 3D tools. Below the reconstruction sits a summary line (here four components and six known connections) and six collapsible sections: *Assembly Type Identified*, *Expected Components Missing* (the AssemblyTemplateDB prior), *AI-Ranked Next Component* (the NodeRanker output), *Why This Next Component?* (the GNNExplainer attributions), *Open Assembly Joints Detected* (the octree open-surface findings) and *Suggested Missing-*

Part Shapes. *Predict another* loads a new file, and *Reset All & Log* clears the session.

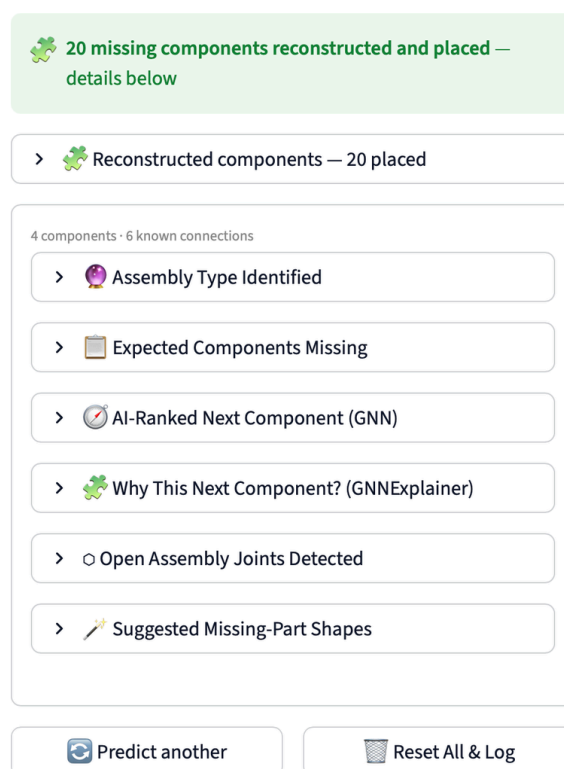


Figure 7.9: Prediction details panel with the reconstruction banner and the six result sections.

7.5 AIDA Engineering Interpretation

The AIDA panel (Figure 7.10) turns the model outputs into an engineering-language report with fixed sections. Section 1 lists components that touch nothing, or that have fewer contacts than sibling instances of the same part; in this run none met either condition. Section 2 lists the links the GNN predicts between existing parts. Here every one carries a confidence of 0.001, and AIDA reports them as very low confidence rather than presenting them as findings. Section 3 reports the octree detector’s open-surface fractions (34%, 30% and 29% for Bodies 2, 3 and 4). Section 4 lists the suggested missing-part shapes together with a location fit and a shape confidence for each: the washer, nut and bolt are retrieved from the part bank at 99–100% on both scores, while the long shaft is generated by the conditional VAE with a 97% shape confidence but only 34% location fit. The last item is worth noting. It shows the VAE fallback being used for a non-fastener type in the live application, and it shows that for the generated part the weak point is the placement, not the shape. A closing recommendation summarises the findings.

© AIDA explains Gemini AI · engineering interpretation of GNN predictions

=== 1. Not Assembled / Not Properly Mated ===

- All existing components in the assembly are currently considered properly mated or fully constrained.
- There are no identified components that are completely unattached or improperly constrained within the current assembly state.

=== 2. AI Predicted Missing Links ===

- The GNN predicts a potential connection between the Tool Holder and DIN 125-2 A B 003, but with a very low confidence of 0.001.
- A connection between the Base Plate and Clamping Nut is predicted with a very low confidence of 0.001.
- A reciprocal connection between the Clamping Nut and Base Plate is predicted with a very low confidence of 0.001.

=== 3. Open Assembly Joints Detected ===

- Body 2 has a significant open surface area of 34%, indicating potential unconstrained mating faces.
- Body 3 has an open surface area of 30%, suggesting it requires further connections.
- Body 4 has an open surface area of 29%, indicating it is likely under-constrained or awaiting additional components.

=== 4. Suggested Missing-Part Shapes ===

- A Washer is suggested, retrieved from the part bank, with 100% location fit and 100% shape confidence.
- A Nut is suggested, retrieved from the part bank, with 100% location fit and 100% shape confidence.
- A Bolt is suggested, retrieved from the part bank, with 99% location fit and 99% shape confidence.
- A Long_shaft is suggested, AI-generated (VAE), with 34% location fit and 97% shape confidence.

=== Overall Recommendation ===

The assembly appears to have several open mating surfaces, particularly on Body 2, Body 3, and Body 4, despite the GNN showing very low confidence in specific missing links between existing parts. The high confidence suggestions for standard fasteners (washers, nuts, bolts) and shafts indicate the assembly likely requires additional components to achieve full constraint and functionality.

Figure 7.10: AIDA panel: Gemini-generated engineering interpretation of the GNN predictions, open-surface findings and suggested shapes.

Chapter 8

Testing and Validation

This chapter describes how the system’s correctness was established, separately from how its performance was measured. The distinction matters: a model can post a strong AUC while a defect upstream of it quietly corrupts the inputs, as Section 6.6 demonstrates.

8.1 Reproducibility Protocol

Every reported metric must be reproducible from the same corpus snapshot. This was not true early in the project, and making it true required fixing a real defect.

`get_splits()` seeds the per-graph edge split, but it originally seeded only the PyTorch generator and not Python’s own. Because `RandomLinkSplit` draws on both, re-evaluating an identical checkpoint on an identical fold produced a slightly different AUC on each run. The fix seeds both generators per *(fold, graph)* pair. Verification was direct: the same checkpoint, re-evaluated on the same fold, now returns a bit-identical AUC.

Table 8.1: Reproducibility controls.

Control	Mechanism
Split determinism	Both RNGs seeded per (fold, graph) in <code>get_splits()</code>
Feature-set versioning	Cache invalidated whenever the node-vector width changes
Configuration capture	Every checkpoint stores the <code>config.yaml</code> it trained under
Metric provenance	Checkpoints embed their own CV summary and test metrics
Batch determinism	Sampler shuffle seeded from the resumed epoch, not a reset counter

The feature-set versioning control deserves a note, because it was added after a near-miss. The per-graph cache is keyed by filename alone. When the node vector grew from 22 to 34 dimensions, 235 graphs cached under the old width were about to be silently mixed into a dataset built under the new one, producing a corpus of inconsistent tensor widths. A version stamp on the feature set now invalidates the cache whenever the vector

changes.

8.2 Validation of the Graph Construction Pipeline

The pipeline is validated at three levels.

Structural validation. Every parsed graph is checked for node/edge count consistency, correct feature-vector width, and absence of self-loops. Graphs failing any check are rejected rather than repaired, on the principle that a silently-corrected graph is worse than a missing one.

Classification validation. Component-type assignment was audited against manual inspection, which is how both geometry-classification bugs in Section 5.6 were caught — the fastener-orientation error and the merged-hole heuristic. Neither produced an exception; both were visible only by looking at what the classifier actually output for known parts.

Statistical validation. The exploratory pass described in Section 6.6 examines the distribution of every feature across the whole corpus. This is what surfaced the dead SDF feature, and it is the only one of the three levels that could have.

8.3 Model Validation

Phase 1 is validated under true five-fold cross-validation with chance-level AP reported alongside AP, so that dataset imbalance can never be mistaken for ranking skill. Phase 2 is validated against a corpus-wide majority-class baseline computed from true type frequency rather than from the leave-one-out sample pool — the latter makes the baseline a moving target whenever sampling parameters change, and is retained only for continuity with older logs. Phase 3 selects its checkpoint by validation IoU and Chamfer distance rather than training loss.

8.4 Checkpoint Promotion Gating

A newly trained checkpoint does not automatically replace the serving model. A promotion gate compares candidate metrics against the incumbent and requires both AUC-ROC and average precision to improve, or to remain within a defined tolerance.

The gate itself contained a defect for part of the project’s history, described in Section 5.6: comparing metrics as a lexicographic tuple meant Python’s short-circuiting silently ignored AP whenever AUC differed. A checkpoint that improved AUC while

regressing AP would be promoted with no warning. The corrected gate evaluates both metrics explicitly.

This class of defect — one that produces no error, no warning and no anomalous number, and merely makes a reported quantity decorative — is the hardest to find, and is the reason the ablation protocol in Section 6.5 reports negative and inconclusive results rather than only successful ones.

8.5 System Testing

End-to-end testing runs real STEP files through the deployed application. A representative test asset is a tool-post assembly with its bolts deliberately removed, which exercises the full detect $\rightarrow$ rank $\rightarrow$ generate $\rightarrow$ explain path and has a known correct answer.

Table 8.2: End-to-end pipeline timing, typical four-body assembly.

Stage	Share of latency	Notes
STEP parse and meshing	Dominant	Bounded by the parse-timeout policy
Feature extraction	Moderate	Hole analysis is the costliest component
Encoder forward pass	Negligible	One pass, shared by all heads
Task heads	Negligible	Small parameter counts
Explanation generation	Moderate	External API round-trip
Total	≈ 24 s	Against a <30 s requirement (NFR-1)

The distribution is itself informative: geometry processing dominates, and the neural components are effectively free by comparison. Any future latency work belongs in the parser, not the model.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

This project set out to close a specific gap: CAD systems track geometry faithfully but offer no judgement about whether an assembly is complete. The result is an end-to-end graph neural network system that treats a mechanical assembly as an attributed graph and addresses assembly completion in three phases sharing a single frozen encoder.

All objectives stated in Chapter 1 were met. Phase 1 detects missing components at a mean AUC-ROC of 0.9206 (± 0.0121) and mean AP of 0.9020 under true five-fold cross-validation, against targets of 0.85 and 0.82. Phase 2 recommends the next component type at Hit@5 0.9101 and MRR 0.6102 against targets of 0.70 and 0.60, with Hit@1 of 0.4093 genuinely above the 0.3442 majority-class baseline. Phase 3 generates placeable geometry at IoU 0.6459 and Chamfer distance 0.0377, comfortably clearing targets of 0.35 and 0.08.

Beyond the headline metrics, the project delivers four contributions with no direct analogue in the surveyed CAD-generation literature: an octree-based open-surface detector that locates candidate mating joints from raw mesh geometry without a CAD kernel; a component-type frequency prior that remains useful when a single body is uploaded with no graph context; a GNNExplainer-based explanation layer that renders predictions in engineering language; and a working interactive application running the complete pipeline against user-uploaded STEP files in approximately 24 seconds.

The project also delivers a curated corpus of 749 industrial CAD assemblies balanced across seven categories, together with the parsing pipeline that converts them into attributed graphs — an artefact that outlives this particular model.

If one lesson generalises beyond this specific system, it is the one in Section 6.2: across thirty-nine tracked revisions, the improvements that mattered came from correcting bro-

ken data and broken metrics, not from searching model space.

9.2 Limitations

Three limitations are worth stating plainly.

First, NodeRanker’s majority-class bias means Hit@1 performance, while a genuine improvement over baseline, is not evenly distributed across component types. `washer` sits at zero and `thin_plate` at 0.024. Class-balanced sampling or loss re-weighting is the natural next step; an earlier attempt at class-balanced BPR sampling weakly reduced the over-prediction of `body` without resolving the underlying collapse.

Second, HybridShapeGenerator’s conditional-VAE fallback is available for every component type, but only the fastener types — bolt, washer, nut — receive the relaxed retrieval threshold, and the fallback was never invoked in the offline evaluation. Its output quality is therefore not covered by the reported IoU and Chamfer figures, even though the live application does call it for non-fastener types: the long shaft in Figure 7.10 was generated this way, with a low location fit (34%). Measuring the fallback on those types, and re-running the retrieval-threshold analysis with them included, is future work.

Third, the SDF and hole-detection findings in Section 6.6 are open engineering issues rather than fixed ones. Confirming and repairing the SDF ray-cast, and manually auditing the bolt/washer hole-detection false-positive rate against ground-truth STEP geometry, are concrete near-term priorities before the next full corpus retrain.

More broadly, the corpus — while curated across seven realistic industrial categories — is domain-specific to fastening and flow-control mechanisms. No claim is made about generalisation beyond that domain.

9.3 Future Work

1. **Address the ranking-head class bias** through class-balanced sampling, focal or re-weighted ranking loss, or a richer context representation. The mean-pooled context vector of Equation 4.2 is a prime suspect: averaging over all known nodes may simply wash out the signal that distinguishes a washer-shaped gap from a body-shaped one.
2. **Repair the dead SDF feature** and re-run the full corpus pipeline, then measure whether restoring two genuinely informative dimensions produces a gain comparable to the earlier hole-count/joint-type repair.

3. **Validate the generative fallback** on the non-fastener types the live application already calls it for, and raise voxel resolution above 32^3 , which currently bounds achievable detail.
4. **Extend corpus coverage** to a wider range of mechanical assembly types — gearboxes, linkages, weldments — to test whether the approach generalises past fastening and flow-control mechanisms.
5. **Predict joint constraints, not only components**, so that a suggested part arrives with its mating constraints already proposed and can be written back into a native CAD file.
6. **Conduct a user study** with practising mechanical designers to measure whether the explanation layer actually improves trust calibration, which this project asserts but has not empirically tested.

Bibliography

- [1] Xu, X., Willis, K.D.D., Lambourne, J.G., Cheng, C.-Y., Jayaraman, P.K., and Furukawa, Y., “BrepGen: A B-Rep Generative Diffusion Model with Structured Latent Geometry”, *ACM SIGGRAPH 2024 Conference Proceedings*, 2024.
- [2] Jayaraman, P.K., Lambourne, J.G., Desai, N., Willis, K.D.D., Sanghi, A., and Morris, N.J.W., “SolidGen: An Autoregressive Model for Direct B-Rep Synthesis and Editing”, *ACM Transactions on Graphics*, vol. 43, 2024.
- [3] Wang, X., Guo, R., and Fu, R., “CAD-GPT: Synthesising CAD Construction Sequences with Spatial-Reasoning Language Models”, *arXiv preprint arXiv:2501.09803*, 2025.
- [4] “Can Graph Neural Networks Learn Link Heuristics?”, *arXiv preprint arXiv:2411.14711*, 2024.
- [5] “Heterogeneous Graph Contrastive Learning”, *arXiv preprint arXiv:2303.00995*, 2023.
- [6] Kipf, T.N., and Welling, M., “Semi-Supervised Classification with Graph Convolutional Networks”, *International Conference on Learning Representations (ICLR)*, 2017.
- [7] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., and Bengio, Y., “Graph Attention Networks”, *International Conference on Learning Representations (ICLR)*, 2018.
- [8] Hamilton, W.L., Ying, R., and Leskovec, J., “Inductive Representation Learning on Large Graphs”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [9] Busbridge, D., Sherburn, D., Cavallo, P., and Hammerla, N.Y., “Relational Graph Attention Networks”, *arXiv preprint arXiv:1904.05811*, 2019.
- [10] Koch, S., Matveev, A., Jiang, Z., Williams, F., Artemov, A., Burnaev, E., Alexa, M., Zorin, D., and Panozzo, D., “ABC: A Big CAD Model Dataset for Geometric Deep

- Learning”, *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [11] Mo, K., Zhu, S., Chang, A.X., Yi, L., Tripathi, S., Guibas, L.J., and Su, H., “Part-Net: A Large-Scale Benchmark for Fine-Grained and Hierarchical Part-Level 3D Object Understanding”, *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019.
- [12] Ying, R., Bourgeois, D., You, J., Zitnik, M., and Leskovec, J., “GNNExplainer: Generating Explanations for Graph Neural Networks”, *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [13] Fey, M., and Lenssen, J.E., “Fast Graph Representation Learning with PyTorch Geometric”, *arXiv preprint arXiv:1903.02428*, 2019.
- [14] Rendle, S., Freudenthaler, C., Gantner, Z., and Schmidt-Thieme, L., “BPR: Bayesian Personalized Ranking from Implicit Feedback”, *Conference on Uncertainty in Artificial Intelligence (UAI)*, 2009.
- [15] Borah, S., and Borah, B., “Predictive Error Expansion Based Reversible Data Hiding Scheme for Three-Dimensional Mesh Models”, *Multimedia Tools and Applications*, 2020.
- [16] Kingma, D.P., and Welling, M., “Auto-Encoding Variational Bayes”, *International Conference on Learning Representations (ICLR)*, 2014.
- [17] Choy, C.B., Xu, D., Gwak, J., Chen, K., and Savarese, S., “3D-R2N2: A Unified Approach for Single and Multi-view 3D Object Reconstruction”, *European Conference on Computer Vision (ECCV)*, 2016.
- [18] Grover, A., and Leskovec, J., “node2vec: Scalable Feature Learning for Networks”, *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.